

C# / .NET Learning Roadmap

Topic Notes — Section 1: C# Language Fundamentals (Part 2)

Topics: 1.11 through 1.23

Date: 29 June 2026

Section 1 — C# Language Fundamentals (continued)

This document covers topics 1.11 through 1.23, continuing from the previous PDF which covered 1.1 through 1.10.

1.11. Methods — Parameters, Return Types, Overloading, ref/out/in

Builds on: 1.4 (variables and types — parameters and return types are typed), 1.7 (control flow — methods contain it), 1.8 (loops — methods wrap them).

What are methods and why do they exist?

Without methods, every piece of logic would need to be written out in full every time it's needed. Methods let you name a block of code, give it inputs, and reuse it. They're the primary unit of code organisation in C#.

Two problems they solve:

- Reuse — write once, call many times
- Abstraction — the caller doesn't need to know how something works, just what to call

Anatomy of a method

```
public decimal CalculateTax(decimal amount, decimal rate)
{
    return amount * rate / 100;
}
```

Breaking it down:

```
public          -> access modifier (who can call this)
decimal         -> return type (what it gives back)
CalculateTax    -> method name
(decimal amount, decimal rate) -> parameter list
{ return ... }  -> method body
```

Return types

A method either returns a value or it doesn't.

```
// Returns a value -- caller gets something back
public int Add(int a, int b)
{
    return a + b;
}
```

```
// Returns nothing -- void means no return value
public void LogMessage(string message)
{
    Console.WriteLine($"[LOG] {message}");
    // no return statement needed (or return; with no value)
}
```

Every code path must return a value when return type is non-void:

```
public string GetGrade(int score)
{
    if (score >= 90) return "A";
    if (score >= 80) return "B";
    // compile error -- what if score < 80?
    // compiler requires all paths to return
}

// Fixed:
public string GetGrade(int score)
{
    if (score >= 90) return "A";
    if (score >= 80) return "B";
    return "F";    // all paths covered
}
```

Parameters — inputs to a method

Value parameters (default) — a copy is passed in:

```
public void Double(int number)
{
    number *= 2;           // modifies the LOCAL copy
    Console.WriteLine(number); // prints doubled value
}

int x = 5;
Double(x);
Console.WriteLine(x);    // still 5 -- original unchanged
```

This follows directly from value type copy semantics (1.4).

Reference type parameters — the reference is copied, but both point to the same object:

```
public void AddItem(List<string> list)
{
    list.Add("new item");    // mutates the SAME list
}

var myList = new List<string> { "one" };
AddItem(myList);
Console.WriteLine(myList.Count);    // 2 -- list was modified
```

ref — pass by reference

Forces the method to work on the original variable, not a copy. Changes made inside the method affect the caller's variable. The variable must be initialised before passing.

```
public void Double(ref int number)
{
    number *= 2;
}

int x = 5;
Double(ref x);           // ref required at call site too
Console.WriteLine(x);    // 10 -- original was changed
```

ref is rare in typical .NET application code. You'll see it in performance-sensitive scenarios and interop. Don't reach for it by default.

out — return additional values

out lets a method return multiple values through parameters. The variable does NOT need to be initialised before passing (the method is responsible for assigning it). The method MUST assign the out parameter before returning.

```
public bool TryDivide(int a, int b, out int result)
{
    if (b == 0)
    {
        result = 0;    // must assign before returning
        return false;
    }
    result = a / b;
    return true;
}

// Call site:
if (TryDivide(10, 2, out int quotient))
{
    Console.WriteLine(quotient);    // 5
}
```

You already know this pattern — `int.TryParse` uses out (section 1.5). That's the canonical example of out in .NET.

Discard — when you don't need the out value:

```
// You just want to know if it's a valid int, not the value
if (int.TryParse(input, out _))
{
    Console.WriteLine("Valid integer");
}
```

`_` is the discard — tells the compiler you're intentionally ignoring the out value.

in — read-only reference

Passes by reference (no copy) but the method cannot modify it. Used for performance with large structs — avoids copying without risking mutation. Rare in everyday API code.

```
public double CalculateDistance(in Point a, in Point b)
{
    // a and b are passed by reference, but cannot be modified here
    return Math.Sqrt(Math.Pow(b.X - a.X, 2) + Math.Pow(b.Y - a.Y, 2));
}
```

Method overloading

Multiple methods can share the same name if their parameter lists differ. The compiler picks the right one based on the arguments you pass.

```
public class Logger
{
```

```

public void Log(string message)
{
    Console.WriteLine(message);
}

public void Log(string message, string level)
{
    Console.WriteLine($"[{level}] {message}");
}

public void Log(Exception ex)
{
    Console.WriteLine($"ERROR: {ex.Message}");
}
}

var logger = new Logger();
logger.Log("Started");           // calls first overload
logger.Log("Started", "INFO");   // calls second overload
logger.Log(new Exception("fail")); // calls third overload

```

Rules for valid overloads:

- Must differ in number of parameters, types of parameters, or order of types
- Return type alone is NOT enough to distinguish overloads

```

// Invalid -- same signature, just different return type
public int Process(string input) { ... }
public string Process(string input) { ... } // compile error

```

Where overloading appears in .NET: Console.WriteLine has 18 overloads. string.Split has multiple. Every constructor overload you write is overloading.

Expression-bodied methods

For single-expression methods, a shorter syntax using `=>`:

```

// Full syntax:
public int Add(int a, int b)
{
    return a + b;
}

// Expression-bodied:
public int Add(int a, int b) => a + b;

// Works for void too:
public void Log(string msg) => Console.WriteLine(msg);

```

Common in .NET codebases for simple property getters and short utility methods.

Static vs instance methods

Instance methods operate on a specific object — they have access to this:

```

public class Order
{
    public decimal Amount { get; set; }
}

```

```

    public decimal CalculateTax()    // instance method
    {
        return Amount * 0.18m;      // accesses instance data
    }
}

```

```

var order = new Order { Amount = 1000 };
order.CalculateTax();    // called on an instance

```

Static methods belong to the type itself, not any instance. No this, no instance data:

```

public class MathHelper
{
    public static int Square(int n) => n * n;
}

```

```

MathHelper.Square(5);    // called on the type, not an instance

```

In ASP.NET Core, service methods are instance methods. Utility/helper methods (parsing, formatting, validation) are often static.

Realistic scenario — service method in a .NET API

```

public class OrderService
{
    private readonly IOrderRepository _repo;

    public OrderService(IOrderRepository repo)
    {
        _repo = repo;
    }

    // Returns value -- found order or null
    public Order? GetById(int id)
    {
        return _repo.FindById(id);
    }

    // void -- performs action, returns nothing
    public void CancelOrder(int id)
    {
        var order = _repo.FindById(id);
        if (order == null) return;

        order.Status = "Cancelled";
        _repo.Save(order);
    }

    // out -- returns success + reason
    public bool TryPlaceOrder(OrderDto dto, out string error)
    {
        if (dto.Amount <= 0)
        {
            error = "Amount must be positive";
            return false;
        }
    }
}

```

```

        _repo.Create(dto);
        error = string.Empty;
        return true;
    }

    // Overload -- place order with or without a discount code
    public bool TryPlaceOrder(OrderDto dto, string discountCode, out string error)
    {
        dto.Amount = ApplyDiscount(dto.Amount, discountCode);
        return TryPlaceOrder(dto, out error);    // calls the other overload
    }

    private static decimal ApplyDiscount(decimal amount, string code)
        => code == "SAVE10" ? amount * 0.9m : amount;
}

```

Common Misconceptions

- **ref and out are the same.** ref requires the variable to be initialised before passing — the method may or may not modify it. out doesn't require initialisation — the method is contractually required to assign it before returning. Different intent, different rules.
- **Overloading is resolved at runtime.** No — the compiler picks the overload at compile time based on the static types of the arguments. This is called static dispatch.
- **void methods can't have a return statement.** They can — return; (no value) is valid and useful for early exit. What they can't do is return someValue;.
- **Passing a reference type to a method lets the method reassign it.** No — the method gets a copy of the reference. It can mutate the object the reference points to, but reassigning the parameter doesn't affect the caller. To allow reassignment, you'd need ref.
- **Method names can be anything as long as they're unique.** They must follow identifier rules, and by C# convention, method names are PascalCase (CalculateTax, not calculateTax or calculate_tax).

Practice Problem

Write a class StringUtils with these methods:

- Truncate(string text, int maxLength) — returns the string trimmed to maxLength characters, appending "..." if it was truncated. If text is null or empty, return it as-is.
- An overload Truncate(string text, int maxLength, string suffix) — same behaviour but lets the caller specify the suffix instead of defaulting to "...".

For example:

- Truncate("Hello World", 5) -> "Hello..."
- Truncate("Hi", 10) -> "Hi"
- Truncate("Hello World", 5, ">>") -> "Hello>>"

Self-check — you should now be able to explain:

- The difference between ref and out and give a real use case for out

- Why passing a List to a method lets it mutate the list but reassigning the parameter doesn't affect the caller
- What overloading is and what the compiler uses to pick between overloads
- When to use void vs a return type
- What the `_` discard does in `TryParse(input, out _)`

1.12. Optional and Named Parameters

Builds on: 1.11 — these are extensions to how method parameters work. You need to understand basic parameters before optional or named ones make sense.

What problem do these solve?

Without optional parameters, if you want to support calling a method with different numbers of arguments, you need overloads:

```
// Without optional parameters -- three overloads for variations:
public void SendEmail(string to, string subject) { ... }
public void SendEmail(string to, string subject, string body) { ... }
public void SendEmail(string to, string subject, string body, bool isHtml) { ... }
```

Optional parameters let you collapse these into one method. Named parameters let callers be explicit about which argument maps to which parameter — especially useful when several have the same type.

Optional Parameters

A parameter is optional when it has a default value. If the caller doesn't provide it, the default is used.

```
public void SendEmail(
    string to,
    string subject,
    string body = "",
    bool isHtml = false)
{
    // body and isHtml are optional
}

// All of these are valid calls:
SendEmail("s@zoho.com", "Hello");
SendEmail("s@zoho.com", "Hello", "Hi there");
SendEmail("s@zoho.com", "Hello", "<b>Hi</b>", true);
```

Rules:

- Optional parameters must come after all required parameters
- Default values must be compile-time constants — literals, null, default, const values
- You cannot use runtime values (like `new List()` or `DateTime.Now`) as defaults

```
// Invalid -- optional before required
public void Bad(string optional = "x", string required) { }
```



```
// Invalid -- runtime value as default
public void Bad(List<string> items = new List<string>()) { }

// Valid -- null as default for reference types
public void Good(List<string> items = null) { }
```

Named Parameters

Named parameters let you specify which parameter you're providing by name, regardless of position.

```
public void CreateUser(string name, string email, string role = "Viewer", bool isActive = true)
{
    // ...
}

// Positional (normal):
CreateUser("Sathish", "s@zoho.com", "Admin", true);

// Named -- explicit and readable:
CreateUser(name: "Sathish", email: "s@zoho.com", role: "Admin", isActive: true);

// Named -- lets you skip optional parameters and provide only what you need:
CreateUser("Sathish", "s@zoho.com", isActive: false);
// role gets its default "Viewer", only isActive is overridden
```

Skipping optional parameters is the key practical value. Without named parameters, you can't skip role and provide isActive — you'd have to provide both positionally. Named parameters let you reach in and set only the ones you care about.

Mixing positional and named

You can mix, but positional arguments must come first:

```
// Positional first, then named:
CreateUser("Sathish", "s@zoho.com", isActive: false);

// Named before positional (pre C# 7.2):
CreateUser(name: "Sathish", "s@zoho.com"); // error in older C#
```

From C# 7.2 onwards, named arguments can appear in any position as long as they're in the correct order relative to other named arguments, but the safe habit is: positional first, named after.

Where this appears in real .NET code

In your own service/utility methods:

```
public PagedResult<Order> GetOrders(
    int pageNumber = 1,
    int pageSize = 20,
    string sortBy = "CreatedAt",
    bool ascending = false)
{
    // ...
}
```

```
// Most callers just want page 2 with defaults:
GetOrders(pageNumber: 2);
```

```
// Some want custom page size:
GetOrders(pageNumber: 1, pageSize: 50);
```

Named parameters in test code — extremely common:

```
// Without named params -- hard to read what each bool means:
var order = CreateTestOrder("ORD001", true, false, true);
```

```
// With named params -- self-documenting:
var order = CreateTestOrder(
    id: "ORD001",
    isPaid: true,
    isCancelled: false,
    isShipped: true);
```

Optional parameters vs overloading — when to use which

Situation	Use
Variations in how many arguments you provide	Optional parameters
Fundamentally different behaviour per sig	Overloads
Different types as input	Overloads
Just different defaults	Optional parameters

```
// Good use of optional -- same behaviour, just configurable:
public string FormatDate(DateTime date, string format = "yyyy-MM-dd", bool utc = false)

// Better as overloads -- different behaviour:
public Order GetOrder(int id) { ... } // get by ID
public Order GetOrder(string reference) { ... } // get by reference string
```

One gotcha — default values are baked into the caller

This is subtle but important. When you compile a call to a method with optional parameters, the default value is embedded into the calling assembly at compile time — not read from the method's assembly at runtime.

```
// In LibraryA.dll:
public void DoSomething(int timeout = 30) { }
```

```
// In YourApp.dll -- compiled against LibraryA:
DoSomething(); // compiler embeds: DoSomething(30)
```

If LibraryA later changes the default to 60 and you release a new version of LibraryA without recompiling YourApp, YourApp still uses 30. This matters in library/SDK development — for public APIs, overloads are safer than optional parameters because changing a default is a breaking change in disguise.

For application code (where everything is compiled together), this rarely matters.

Common Misconceptions

- **Optional parameters must have a value provided if the type is non-nullable.** No — you can default to null for any reference type regardless of nullability. `string name = null` is a valid

optional parameter default even with nullable reference types enabled.

- **Named parameters are only useful for optional parameters.** Named parameters work on any method — even ones with no optional parameters. They're independently useful for readability when a method takes multiple same-type parameters.
- **Skipping optional parameters in the middle is possible positionally.** It's not. You can only skip by using named parameters. Positionally, you must provide every argument up to and including the last one you want to set.
- **Changing a default value in a method is a safe non-breaking change.** Not for library code — callers compiled against the old default will continue using it until they recompile. Treat default value changes as breaking changes in library/SDK scenarios.

Practice Problem

Write a method `BuildConnectionString` that constructs a database connection string with these parameters:

- server (required)
- database (required)
- port (optional, default 1433)
- timeout (optional, default 30)
- encrypt (optional, default true)

It should return a string like: `"Server=myserver,1433;Database=mydb;Connection Timeout=30;Encrypt=True"`

Then show three different ways to call it using a mix of positional and named arguments. Try it before moving on.

Self-check — you should now be able to explain:

- What rule governs where optional parameters must appear in the parameter list
- How to skip a middle optional parameter without providing the ones after it
- Why changing a default value can be a breaking change for library consumers
- When to prefer optional parameters over overloads and vice versa
- What named parameters give you even when all parameters are required

1.13. Exception Handling — try/catch/finally, Custom Exceptions

Builds on: 1.11 (methods — exceptions propagate up the call stack through methods), 1.4 (types — exceptions are objects with a type hierarchy).

What are exceptions and why does this mechanism exist?

In any real program, things go wrong at runtime — a file doesn't exist, a database is unreachable, a user passes invalid data, a network times out. Without a structured way to handle these failures, your program would either crash silently or require every method to return

success/failure codes that callers can easily ignore.

Exceptions give you a structured, impossible-to-silently-ignore mechanism for signalling and handling runtime failures. When something goes wrong, an exception is thrown. If nothing catches it, the program crashes with a meaningful error. If something catches it, you decide what to do.

The call stack and exception propagation

When an exception is thrown, .NET walks back up the call stack looking for a matching catch block. If it finds one, execution jumps there. If it reaches the top of the stack without finding one, the program terminates.

```
Main()
  +-- ProcessOrder()
    +-- ChargePayment()
      +-- CallPaymentApi() <- exception thrown here
```

If CallPaymentApi throws and has no catch, .NET looks in ChargePayment. Not there? Looks in ProcessOrder. Not there? Looks in Main. Not there? Program crashes.

This propagation means you can handle exceptions at the right level — not necessarily where they occur.

try / catch

```
try
{
    // code that might throw
    int result = int.Parse("abc"); // throws FormatException
    Console.WriteLine(result);      // never reached
}
catch (FormatException ex)
{
    // handles FormatException specifically
    Console.WriteLine($"Invalid format: {ex.Message}");
}
```

The catch block only runs if the specified exception type (or a subtype) is thrown. Code after the throw inside try is skipped — execution jumps directly to catch.

Catching multiple exception types

```
try
{
    var data = File.ReadAllText("config.json");
    var config = JsonSerializer.Deserialize<Config>(data);
    int port = int.Parse(config.Port);
}
catch (FileNotFoundException ex)
{
    Console.WriteLine($"Config file missing: {ex.FileName}");
}
catch (JsonException ex)
{
}
```

```

        Console.WriteLine($"Invalid JSON: {ex.Message}");
    }
    catch (FormatException ex)
    {
        Console.WriteLine($"Invalid port number: {ex.Message}");
    }

```

Catch blocks are evaluated top to bottom — first matching type wins. Order matters when types are related (more specific types must come before less specific ones).

The Exception hierarchy — why order matters

All exceptions inherit from `System.Exception`. Common hierarchy:

```

Exception
+-- SystemException
|   +-- NullReferenceException
|   +-- IndexOutOfRangeException
|   +-- InvalidCastException
|   +-- OverflowException
|   +-- IOException
|       +-- FileNotFoundException
+-- ApplicationException (rarely used directly)
+-- ... your custom exceptions

```

If you catch `Exception` before `FileNotFoundException`, the `FileNotFoundException` catch is unreachable:

```

// Wrong order -- FileNotFoundException never caught specifically:
catch (Exception ex) { ... }           // catches everything
catch (FileNotFoundException ex) { ... } // never reached

// Correct -- specific first, general last:
catch (FileNotFoundException ex) { ... }
catch (Exception ex) { ... }

```

Catch without a type — catch-all

```

catch (Exception ex)
{
    // catches ANY exception
    Console.WriteLine($"Unexpected error: {ex.Message}");
}

```

Use this as a last resort at the top level (e.g. in `Program.cs` or a middleware) to prevent complete crashes. Don't use it everywhere — catching `Exception` blindly hides bugs.

finally — always runs

The `finally` block runs regardless of whether an exception was thrown or not. Used for cleanup that must happen no matter what.

```

SqlConnection connection = null;

try
{
    connection = new SqlConnection(connectionString);

```

```

        connection.Open();
        // ... do work
    }
    catch (SqlException ex)
    {
        Console.WriteLine($"DB error: {ex.Message}");
    }
    finally
    {
        connection?.Close(); // always closes, even if exception occurred
    }

```

In modern C#, using statements (section 8.2) replace most finally patterns for resource cleanup. But finally remains useful for logging, resetting state, or any cleanup that isn't about disposable resources.

Re-throwing exceptions

Sometimes you catch an exception to log it or add context, then re-throw it for the caller to handle:

```

try
{
    _repo.Save(order);
}
catch (SqlException ex)
{
    _logger.LogError(ex, "Failed to save order {OrderId}", order.Id);
    throw; // re-throw the SAME exception, preserving the original stack trace
}

```

Critical distinction:

```

throw; // preserves original stack trace -- use this
throw ex; // resets stack trace to this line -- loses where it originated

```

Always use bare throw; when re-throwing. throw ex; destroys the diagnostic information that tells you where the error actually happened.

when — exception filters

Catch only when a condition is true:

```

try
{
    var response = await httpClient.GetAsync(url);
    response.EnsureSuccessStatusCode();
}
catch (HttpRequestException ex) when (ex.StatusCode == HttpStatusCode.NotFound)
{
    return NotFound();
}
catch (HttpRequestException ex) when (ex.StatusCode == HttpStatusCode.Unauthorized)
{
    return Unauthorized();
}
catch (HttpRequestException ex)

```

```

{
    _logger.LogError(ex, "HTTP request failed");
    return StatusCode(500);
}

```

The when clause lets you handle the same exception type differently based on its content — without nested if statements inside a single catch block.

Custom exceptions

Create custom exceptions when you need to signal a domain-specific error that callers can catch specifically, or carry additional context beyond what built-in exceptions provide.

```

// Custom exception -- inherit from Exception (or a relevant subtype)
public class OrderNotFoundException : Exception
{
    public int OrderId { get; }

    public OrderNotFoundException(int orderId)
        : base($"Order {orderId} was not found")
    {
        OrderId = orderId;
    }

    // Always provide this constructor for serialization compatibility:
    public OrderNotFoundException(int orderId, Exception inner)
        : base($"Order {orderId} was not found", inner)
    {
        OrderId = orderId;
    }
}

```

Throwing it:

```

public Order GetOrder(int id)
{
    var order = _repo.FindById(id);

    if (order == null)
        throw new OrderNotFoundException(id);

    return order;
}

```

Catching it specifically:

```

try
{
    var order = _orderService.GetOrder(orderId);
    return Ok(order);
}
catch (OrderNotFoundException ex)
{
    return NotFound($"Order {ex.OrderId} not found");
}

```

Exception handling in ASP.NET Core — where it actually lives

In a real .NET API, you don't wrap every controller action in try/catch. Instead, global exception handling middleware catches unhandled exceptions from anywhere in the pipeline:

```
// Program.cs
app.UseExceptionHandler(errorApp =>
{
    errorApp.Run(async context =>
    {
        context.Response.StatusCode = 500;
        await context.Response.WriteAsJsonAsync(new
        {
            error = "An unexpected error occurred"
        });
    });
});
```

Specific catches go in service methods for expected failures — things you anticipate and handle deliberately:

```
public async Task<bool> SendNotification(int userId)
{
    try
    {
        await _emailService.SendAsync(userId);
        return true;
    }
    catch (SmtpException ex)
    {
        _logger.LogWarning(ex, "Email failed for user {UserId}", userId);
        return false;    // graceful degradation -- app continues
    }
}
```

The pattern: handle what you expect at the point it occurs, let everything else bubble up to global handling.

Common Misconceptions

- **I should wrap everything in try/catch.** You shouldn't. Exceptions should be exceptional — wrapping normal control flow in try/catch is slow and hides bugs. Use TryParse, null checks, and validation to prevent predictable failures instead.
- **throw ex and throw are the same.** They're not. throw ex resets the stack trace — you lose the information showing where the exception originated. Always use bare throw when re-throwing.
- **Catching Exception is a safety net I should always have.** At the outermost level (global middleware), yes. Inside individual methods, no — it swallows bugs that should surface.
- **finally runs only if no exception occurred.** The opposite — finally runs whether or not an exception occurred. That's its entire purpose. The only time finally doesn't run is if the process is killed.
- **Custom exceptions should inherit from ApplicationException.** This was the original recommendation but was reversed. Inherit from Exception directly (or the most relevant specific exception type). ApplicationException has no special meaning in modern .NET.

- **Exceptions are the right way to handle all error conditions.** Not always. For expected, recoverable failures (user not found, invalid input), returning null, a result type, or a bool+out pattern (TryX) is often cleaner.

Practice Problem

Write a method `ParseConfig(string json)` that:

- Deserialises the JSON string into a `Config` class (with `Host` and `Port` string properties)
- Throws a custom `InvalidConfigException` if `Host` is null or empty
- Parses `Port` to an `int` — if it fails, throws `InvalidConfigException` with a message saying the port is invalid
- Returns the parsed host and port as a tuple (`string host, int port`)

Also write the `InvalidConfigException` class itself.

Self-check — you should now be able to explain:

- What exception propagation means and how the call stack is involved
- Why `throw` preserves the stack trace but `throw ex` doesn't
- What `finally` guarantees and when it runs
- When to create a custom exception vs use a built-in one
- Why catching `Exception` everywhere is an antipattern but catching it at the global level is correct

1.14. Nullable Types and Nullable Reference Types

Builds on: 1.4 (value vs reference types — nullability rules differ between them), 1.6 (`??` and `?.` operators — you've already used null handling), 1.13 (`NullReferenceException` is what nullable reference types help you prevent).

The null problem

null means "no value" — but in C#, how null works has always been inconsistent between value types and reference types, and it's caused more runtime crashes than almost anything else.

```
string name = null;
Console.WriteLine(name.Length); // NullReferenceException -- runtime crash
```

The compiler didn't warn you. The code compiled fine. It only blew up at runtime. This is the problem that C#'s nullable system exists to solve.

There are two separate but related nullable systems in C# — one for value types, one for reference types.

Part 1 — Nullable Value Types (`int?`, `bool?`, `decimal?`)

Value types (`int`, `bool`, `decimal` etc.) cannot be null by default. This is by design — an `int` always has a value.

But real-world data frequently has "no value" — a database column that allows NULL, an optional form field, an age that wasn't provided. That's what nullable value types solve.

Syntax — append ? to any value type:

```
int age = null;    // compile error -- int cannot be null
int? age = null;   // nullable int -- can hold null or any int
```

int? is shorthand for Nullable. They're identical.

Checking and using nullable value types:

```
int? score = null;

// Check before using:
if (score.HasValue)
{
    Console.WriteLine(score.Value);    // access the actual int
}

// Or use ?? for a fallback:
int display = score ?? 0;              // 0 if null
Console.WriteLine(display);

// GetValueOrDefault -- same as ?? 0:
int safe = score.GetValueOrDefault(); // 0 if null
int safe2 = score.GetValueOrDefault(-1); // -1 if null
```

Where nullable value types appear constantly:

```
// EF Core -- database columns that allow NULL:
public class User
{
    public int Id { get; set; }
    public string Name { get; set; }
    public DateTime? LastLoginDate { get; set; } // nullable -- user may never have logged in
    public int? Age { get; set; }                // nullable -- not always provided
}

// API parameters -- optional query params:
[HttpGet]
public IActionResult GetOrders([FromQuery] int? userId, [FromQuery] DateTime? from)
{
    var orders = userId.HasValue
        ? _repo.GetByUser(userId.Value)
        : _repo.GetAll();

    return Ok(orders);
}
```

Arithmetic and comparison with nullable value types

Nullable value types propagate null — any operation involving null produces null:

```
int? a = 5;
int? b = null;

int? sum = a + b;    // null -- null propagates
int? product = a * b; // null
```

```
// Comparison with null:
Console.WriteLine(b == null);    // true
Console.WriteLine(b > 0);        // false -- null comparisons return false (except ==)
Console.WriteLine(null == null); // true
```

Part 2 — Nullable Reference Types (C# 8+)

This is a fundamentally different feature. Reference types (string, classes, arrays) have always allowed null — there was no way to say "this string is guaranteed not null." C# 8 changed that.

Enabling it — in your .csproj:

```
<Nullable>enable</Nullable>
```

This is on by default in all modern dotnet new templates. Once enabled:

- string means a non-nullable string — the compiler warns if you might assign null
- string? means a nullable string — explicitly opt-in to allowing null

// Nullable reference types enabled:

```
string name = "Sathish";    // non-nullable -- compiler ensures this isn't null
string name = null;         // warning -- non-nullable assigned null

string? name = null;        // explicitly nullable -- null is allowed
string? name = "Sathish";   // also fine
```

The compiler now tracks null state:

```
string? name = GetNameFromDatabase();    // might return null

Console.WriteLine(name.Length);    // warning -- name might be null

// Fix 1 -- null check:
if (name != null)
{
    Console.WriteLine(name.Length);    // safe -- compiler knows it's not null here
}

// Fix 2 -- null conditional:
Console.WriteLine(name?.Length);    // safe -- returns null if name is null

// Fix 3 -- null coalescing:
Console.WriteLine((name ?? "Anonymous").Length);    // safe
```

Non-nullable reference types — what the compiler enforces

```
public class UserService
{
    private readonly IUserRepository _repo;

    // Non-nullable parameter -- compiler warns if null is passed
    public UserService(IUserRepository repo)
    {
        _repo = repo;    // assigned from non-null parameter
    }

    // Non-nullable return type -- must never return null
```

```

public User GetById(int id)
{
    var user = _repo.FindById(id);

    if (user == null)
        throw new UserNotFoundException(id);    // throw instead of returning null

    return user;    // guaranteed non-null
}

// Nullable return type -- explicitly saying this might return null
public User? FindById(int id)
{
    return _repo.FindById(id);    // might be null -- caller must handle it
}
}

```

The naming convention that emerges naturally:

- GetX — returns non-nullable, throws if not found
- FindX or GetXOrDefault — returns nullable, caller handles null

The null-forgiving operator !

Sometimes you know something isn't null but the compiler can't prove it. Use ! to suppress the warning:

```

string? name = GetName();

// You know from business logic this can't be null here,
// but the compiler can't prove it:
Console.WriteLine(name!.Length);    // ! tells compiler "trust me, not null"

```

Use this sparingly — it defeats the purpose of nullable reference types if overused. It's a last resort when the compiler is wrong and you can't restructure the code.

Nullable annotations on properties and parameters

```

public class CreateUserDto
{
    public string Name { get; set; }    // required -- must not be null
    public string Email { get; set; }    // required -- must not be null
    public string? PhoneNumber { get; set; }    // optional -- null is valid
    public string? Bio { get; set; }    // optional
}

```

This makes your intent explicit in code — anyone reading CreateUserDto immediately knows which fields are required and which are optional, without reading documentation.

Combining both nullable systems

```

// Takes a nullable int (value type) and nullable string (reference type)
// Returns a nullable decimal (value type)
public decimal? CalculateDiscount(int? loyaltyPoints, string? promoCode)
{
    if (!loyaltyPoints.HasValue && promoCode == null)

```

```

        return null;    // no discount applicable

decimal discount = 0;

if (loyaltyPoints.HasValue)
    discount += loyaltyPoints.Value * 0.001m;

if (promoCode == "SAVE10")
    discount += 0.10m;

return discount;
}

```

Nullable in EF Core — maps directly to database NULLs

```

public class Product
{
    public int Id { get; set; }
    public string Name { get; set; }           // NOT NULL in DB
    public string? Description { get; set; }   // NULL allowed in DB
    public decimal Price { get; set; }         // NOT NULL in DB
    public decimal? DiscountPrice { get; set; } // NULL allowed in DB
    public DateTime? DiscountExpiry { get; set; } // NULL allowed in DB
}

```

EF Core reads your nullable annotations and creates the corresponding database schema. `string?` -> nullable column. `string` -> NOT NULL column. This is one of the most direct practical benefits of enabling nullable reference types.

Common Misconceptions

- **Nullable reference types prevent `NullReferenceException` at runtime.** They don't — they're compile-time warnings only. If you ignore warnings or use `!`, you can still get `NullReferenceExceptions`. The feature helps you find problems at compile time, not eliminate them at runtime.
- **`string?` and `string` are different types.** They're not — at runtime they're identical. The `?` annotation only exists at compile time for the flow analysis. There's no runtime overhead.
- **Enabling nullable reference types breaks existing code.** It generates warnings, not errors — unless you set `NullableReferenceTypes`. Existing code still compiles; you get a list of places to review.
- **`int?` and `string?` work the same way.** They don't. `int?` is a genuinely different type (Nullable) with `.HasValue` and `.Value`. `string?` is just a compile-time annotation — same type as `string` at runtime.
- **I should use `!` (null forgiving) whenever I get a nullable warning.** This defeats the purpose. The `!` operator is for when the compiler is provably wrong. If you're using it to silence warnings you don't understand, you're hiding real bugs.
- **null and default are always the same.** For reference types, default is null. For value types, default is the zero value (0, false). For `int?`, default is null (not 0).

Practice Problem

You have this class with nullable reference types enabled:

```
public class OrderSummary
{
    public int Id { get; set; }
    public string CustomerName { get; set; }
    public string? PromoCode { get; set; }
    public DateTime? ShippedAt { get; set; }
    public decimal? DiscountAmount { get; set; }
}
```

Write a method `FormatSummary(OrderSummary? order)` that:

- Returns "No order" if order itself is null
- Returns a formatted string like: "Order 42 | Sathish | Promo: SAVE10 | Shipped: 2026-06-29 | Discount: \$10.00"
- If `PromoCode` is null, shows "Promo: None"
- If `ShippedAt` is null, shows "Not shipped"
- If `DiscountAmount` is null or zero, omits the discount section entirely

Use `??`, `?.`, and nullable checks appropriately. Try before moving on.

Self-check — you should now be able to explain:

- The difference between `int?` and `string?` — what each actually is at runtime
- What enabling nullable does and does not do
- Why throw is often the right response in a `GetX` method when the result is null
- How nullable annotations on EF Core entity properties map to the database schema
- When to use the `!` null-forgiving operator and why it should be rare

1.15. Enums

Builds on: 1.4 (value types — enums are value types), 1.7 (switch expressions — enums and switch expressions are a natural pair).

What are enums and why do they exist?

Without enums, representing a fixed set of named options means using either magic strings or magic numbers:

```
// Magic strings -- typo-prone, no compile-time safety:
order.Status = "Shipped";
if (order.Status == "shipped") { ... } // typo -- silent bug, always false

// Magic numbers -- unreadable:
order.Status = 2;
if (order.Status == 2) { ... } // what does 2 mean?
```

Enums replace both with named, compile-time-safe constants:

```
order.Status = OrderStatus.Shipped;
if (order.Status == OrderStatus.Shipped) { ... } // clear and safe
```

The compiler catches typos (`OrderStatus.Shiped` doesn't exist — compile error). The code is self-documenting. The underlying value is still a number, but you never see or use it directly.

Defining and using an enum

```
public enum OrderStatus
{
    Pending,      // 0 by default
    Processing,   // 1
    Shipped,      // 2
    Delivered,    // 3
    Cancelled     // 4
}
```

Values start at 0 and auto-increment. Using it:

```
OrderStatus status = OrderStatus.Pending;

Console.WriteLine(status);           // "Pending" -- ToString() gives the name
Console.WriteLine((int)status);      // 0 -- cast to int to get the number
```

Explicit values

You can assign specific integer values — useful when the number has external meaning (database values, API codes, bit flags):

```
public enum HttpVerb
{
    Get    = 1,
    Post   = 2,
    Put    = 3,
    Patch  = 4,
    Delete = 5
}

public enum Priority
{
    Low      = 10,
    Medium   = 20,
    High     = 30,
    Critical = 100 // gap in values is fine
}
```

Enums and switch expressions — the natural pair

Enums are the ideal input for switch expressions. The compiler knows all possible values — if you miss one and don't have a `discard _`, it warns you:

```
public string GetStatusMessage(OrderStatus status) => status switch
{
    OrderStatus.Pending    => "Your order is being reviewed",
    OrderStatus.Processing => "Your order is being prepared",
    OrderStatus.Shipped    => "Your order is on its way",
    OrderStatus.Delivered  => "Your order has arrived",
    OrderStatus.Cancelled  => "Your order was cancelled",
    _                      => "Unknown status"
```

```
};
```

If you add a new value to the enum later, this switch will produce a warning that you haven't handled the new case — the compiler is actively protecting you.

Converting between enums and strings

Enum to string:

```
OrderStatus status = OrderStatus.Shipped;
string name = status.ToString();           // "Shipped"
string name2 = Enum.GetName(status);      // "Shipped"
```

String to enum — Enum.Parse and Enum.TryParse:

```
// Parse -- throws if invalid:
OrderStatus status = Enum.Parse<OrderStatus>("Shipped"); // OrderStatus.Shipped
OrderStatus bad = Enum.Parse<OrderStatus>("shipped");    // ArgumentException

// TryParse -- safe version:
if (Enum.TryParse<OrderStatus>("Shipped", out OrderStatus result))
{
    Console.WriteLine(result); // OrderStatus.Shipped
}

// Case-insensitive parse:
Enum.TryParse<OrderStatus>("shipped", ignoreCase: true, out OrderStatus result);
```

Enum to int and back:

```
int value = (int)OrderStatus.Shipped; // 2
OrderStatus status = (OrderStatus)2;  // OrderStatus.Shipped

// Danger -- no validation on int cast:
OrderStatus invalid = (OrderStatus)99; // no exception! just a value with no name
Console.WriteLine(invalid);            // "99" -- prints the number, not a name

// Safe check:
if (Enum.IsDefined(typeof(OrderStatus), 99)) // false -- 99 is not a valid value
```

Getting all values

```
// All values of an enum:
var allStatuses = Enum.GetValues<OrderStatus>();

foreach (var status in allStatuses)
{
    Console.WriteLine(status);
}
// Pending, Processing, Shipped, Delivered, Cancelled
```

Useful for seeding dropdowns, building validation lists, or populating select options in an API response.

Flags enums — combining values with bitwise OR

A special use case — when enum values represent independent options that can be combined. Decorate with [Flags] and use powers of 2:


```
[Flags]
public enum Permissions
{
    None      = 0,
    Read      = 1,    // 001
    Write     = 2,    // 010
    Delete    = 4,    // 100
    Admin     = Read | Write | Delete    // 111 = 7
}
```

```
// Combine with |:
Permissions userPerms = Permissions.Read | Permissions.Write;
```

```
// Check with HasFlag:
```

```
bool canRead = userPerms.HasFlag(Permissions.Read);    // true
bool canDelete = userPerms.HasFlag(Permissions.Delete); // false
```

```
Console.WriteLine(userPerms);    // "Read, Write" -- [Flags] makes ToString smart
```

Without [Flags], combining values with | works mathematically but ToString() returns the number (3) instead of "Read, Write". Always add [Flags] when using bitwise combinations.

Enums in ASP.NET Core and EF Core

JSON serialisation — numbers vs strings:

By default, ASP.NET Core serialises enums as numbers in JSON responses. To serialise as strings instead — add this in Program.cs:

```
builder.Services.AddControllers()
    .AddJsonOptions(options =>
    {
        options.JsonSerializerOptions.Converters.Add(
            new JsonStringEnumConverter());
    });
```

EF Core — storing enums. By default EF Core stores enum values as integers. To store as strings:

```
// In your DbContext OnModelCreating:
modelBuilder.Entity<Order>()
    .Property(o => o.Status)
    .HasConversion<string>();
```

Storing as strings is more readable in the database and safer when adding new enum values.

Realistic scenario — order processing pipeline

```
public enum OrderStatus { Pending, Processing, Shipped, Delivered, Cancelled }

public class OrderService
{
    public bool TryAdvanceStatus(Order order, out string error)
    {
        var nextStatus = order.Status switch
        {
            OrderStatus.Pending => OrderStatus.Processing,
```

```

        OrderStatus.Processing => OrderStatus.Shipped,
        OrderStatus.Shipped   => OrderStatus.Delivered,
        OrderStatus.Delivered => null,    // terminal state
        OrderStatus.Cancelled => null,    // terminal state
        _ => null
    };

    if (nextStatus == null)
    {
        error = $"Cannot advance from {order.Status}";
        return false;
    }

    order.Status = nextStatus.Value;
    error = string.Empty;
    return true;
}
}

```

Common Misconceptions

- **Enums are just integers with names.** Conceptually yes, but they're a distinct type — you can't assign an int to an enum variable without an explicit cast. The type system protects you from accidentally mixing unrelated enums.
- **Casting an invalid integer to an enum throws an exception.** It doesn't — `(OrderStatus)99` succeeds silently even if 99 has no defined name. Always use `Enum.IsDefined` if you're casting from untrusted integer values.
- **I can use any values for [Flags] enums.** You must use powers of 2 (1, 2, 4, 8, 16...) for flags to work correctly. Using sequential values (1, 2, 3) will cause `HasFlag` to give wrong results because the bit patterns overlap.
- **Enum.TryParse is case-sensitive by default.** It is — "shipped" won't parse to `OrderStatus.Shipped` without `ignoreCase: true`. Always specify case handling explicitly when parsing user or API input.
- **Adding a new enum value in the middle is safe.** It changes the integer values of subsequent members. If you're storing integers in a database or serialising to integer JSON, this is a breaking change. Always add new values at the end, or use explicit integer assignments.

Practice Problem

Define a [Flags] enum `DayOfWeekFlags` with values for each day of the week (Sunday through Saturday, powers of 2). Then:

- Create a variable representing "Monday, Wednesday, Friday" (a typical meeting schedule)
- Check if Thursday is included — print the result
- Add Sunday to the schedule
- Print the final schedule using `ToString()`

Self-check — you should now be able to explain:

- Why casting an out-of-range integer to an enum doesn't throw and how to guard against it
- Why [Flags] values must be powers of 2 and what HasFlag does
- How to safely parse a string to an enum from user or API input
- How to configure ASP.NET Core to serialise enums as strings in JSON responses
- Why adding enum values in the middle (without explicit values) can be a breaking change

1.16. Structs vs Classes

Builds on: 1.4 (value vs reference types — structs are value types, classes are reference types), 1.11 (methods — both can have them), 1.14 (nullable types — struct? is valid).

What is a struct and why does it exist?

You've been using classes as the default way to define custom types. Structs are an alternative that follows value type semantics — they're copied on assignment, live on the stack (mostly), and don't need garbage collection.

The question isn't "what is a struct" but "when does a struct make more sense than a class." The answer comes down to one thing: does this type represent a small, self-contained value that should behave like a number?

The core difference — value vs reference semantics

```
// Class -- reference semantics:
public class PointClass
{
    public int X { get; set; }
    public int Y { get; set; }
}

// Struct -- value semantics:
public struct PointStruct
{
    public int X { get; set; }
    public int Y { get; set; }
}
```

Assignment behaviour:

```
// Class -- both variables point to same object:
var p1 = new PointClass { X = 1, Y = 2 };
var p2 = p1;
p2.X = 99;
Console.WriteLine(p1.X);    // 99 -- p1 was affected

// Struct -- p2 is an independent copy:
var p3 = new PointStruct { X = 1, Y = 2 };
var p4 = p3;
p4.X = 99;
Console.WriteLine(p3.X);    // 1 -- p3 is unchanged
```

Struct capabilities and limitations

What structs CAN do (same as classes):

- Have properties, fields, methods, constructors
- Implement interfaces
- Have static members
- Be generic (struct Point)

What structs CANNOT do:

- Inherit from another class or struct (no inheritance hierarchy)
- Be used as a base type
- Have a parameterless constructor with custom logic (in older C# — allowed from C# 10)
- Have a finalizer

```
public struct Money
{
    public decimal Amount { get; }
    public string Currency { get; }

    public Money(decimal amount, string currency)
    {
        Amount = amount;
        Currency = currency;
    }

    // Methods are fine:
    public Money Add(Money other)
    {
        if (Currency != other.Currency)
            throw new InvalidOperationException("Currency mismatch");
        return new Money(Amount + other.Amount, Currency);
    }

    public override string ToString() => $"{Amount:F2} {Currency}";
}

// Usage:
var price = new Money(100.00m, "USD");
var tax   = new Money(18.00m, "USD");
var total = price.Add(tax);

Console.WriteLine(total);    // "118.00 USD"
```

Default values — structs are always initialised

A struct variable always has a value — you can never have a null struct (unless you use struct?).

Every field gets its default value:

```
PointStruct p;    // not null -- X = 0, Y = 0
Console.WriteLine(p.X);    // 0
```

This is different from a class where an unassigned variable is null:

```
PointClass p;    // null -- no object
```

```
Console.WriteLine(p.X);    // NullReferenceException
```

When to use a struct — the guidelines

The .NET runtime team guidelines say use a struct when ALL of these are true:

- Small size — typically 16 bytes or less. Larger structs cost more to copy than a reference would.
- Logically a single value — represents one thing (a point, a date range, a money amount, a colour)
- Immutable or rarely mutated — struct mutation has subtle pitfalls
- Short-lived — not stored in large collections or long-lived objects

Good candidates:

```
public struct Coordinate { double Lat; double Lng; }    // two numbers, one concept
public struct DateRange  { DateTime Start; DateTime End; }
public struct Rgb         { byte R; byte G; byte B; }
public struct Money       { decimal Amount; string Currency; }
```

Poor candidates for structs — use classes instead:

```
// Too large, complex lifecycle:
public class Order { ... }    // many fields, DB entity

// Needs inheritance:
public class Animal { ... }   // base for Dog, Cat

// Mutable, shared state:
public class ShoppingCart { ... } // updated by many operations
```

Readonly structs — the right default for modern C#

Mutable structs have a well-known footgun — mutating a struct through a property or in a foreach loop silently operates on a copy:

```
// Mutable struct -- dangerous:
public struct Counter
{
    public int Value;
    public void Increment() { Value++; }
}

var counter = new Counter();
counter.Increment();
Console.WriteLine(counter.Value);    // 1 -- works here

// But in a list -- silently no effect:
var list = new List<Counter> { new Counter() };
list[0].Increment();                // mutates a COPY returned by the indexer
Console.WriteLine(list[0].Value);    // 0 -- original unchanged!
```

The fix — make structs readonly. The compiler then enforces immutability and prevents the footgun entirely:

```
public readonly struct Money
{
```

```

public decimal Amount { get; }
public string Currency { get; }

public Money(decimal amount, string currency)
{
    Amount = amount;
    Currency = currency;
}

// "Mutation" returns a NEW struct -- immutable pattern:
public Money WithAmount(decimal newAmount) => new Money(newAmount, Currency);
}

var price = new Money(100m, "USD");
var discounted = price.WithAmount(90m);    // price unchanged, discounted is new

Console.WriteLine(price.Amount);           // 100
Console.WriteLine(discounted.Amount);      // 90

```

Rule of thumb: if you define a struct, make it readonly unless you have a specific reason not to.

Record structs — C# 10+

C# 10 introduced record struct — combines struct value semantics with record's automatic equality, ToString(), and deconstruction:

```

public record struct Point(int X, int Y);

var p1 = new Point(1, 2);
var p2 = new Point(1, 2);

Console.WriteLine(p1 == p2);           // true -- value equality built in
Console.WriteLine(p1);                 // "Point { X = 1, Y = 2 }" -- ToString built in

var (x, y) = p1;                      // deconstruction built in

```

record struct is the most concise way to define a small immutable value type in modern C#. Prefer it over manual struct definitions for simple data types.

Structs in .NET — what you'll encounter

The BCL uses structs extensively for types that represent single values:

```

DateTime    // date and time -- struct
TimeSpan    // duration -- struct
Guid        // unique identifier -- struct
decimal     // financial number -- struct
int, bool   // all primitives -- struct (technically)
KeyValuePair<TKey, TValue> // dictionary entry -- struct

```

Realistic scenario — using structs for value objects in a domain model

```

public readonly struct Money
{
    public decimal Amount { get; }
    public string Currency { get; }
}

```

```

public Money(decimal amount, string currency)
{
    if (amount < 0) throw new ArgumentException("Amount cannot be negative");
    if (string.IsNullOrEmpty(currency)) throw new ArgumentException("Currency required");
    Amount = amount;
    Currency = currency.ToUpper();
}

public static Money operator +(Money a, Money b)
{
    if (a.Currency != b.Currency) throw new InvalidOperationException("Currency mismatch");
    return new Money(a.Amount + b.Amount, a.Currency);
}

public override string ToString() => $"{Amount:F2} {Currency}";
}

var price = new Money(999.99m, "USD");
var tax    = new Money(179.99m, "USD");
var total = price + tax;
Console.WriteLine(total);    // "1179.98 USD"

```

Structs vs classes — decision summary

Question	Answer -> Use
Does it represent a single value?	Yes -> struct candidate
Is it larger than ~16 bytes?	Yes -> class
Does it need inheritance?	Yes -> class
Stored in large collections?	Yes -> consider class
Should copies be independent?	Yes -> struct
Complex mutable state?	Yes -> class
Short-lived, stack-friendly?	Yes -> struct

When in doubt — use a class. Structs are an optimisation and a semantic choice, not the default.

Common Misconceptions

- **Structs are always on the stack.** Usually, but not always. A struct field inside a class lives on the heap. A struct in a List lives on the heap. The accurate statement is: not separately allocated on the heap as a reference type object.
- **Mutable structs are fine.** They work but have well-known pitfalls — mutating through an indexer, a foreach loop, or a property getter all silently operate on copies. Make structs readonly to eliminate this entire class of bugs.
- **Structs are always faster than classes.** Not for large structs. Copying a 200-byte struct is far more expensive than copying an 8-byte reference. Only small structs get the performance benefit.
- **Structs can't have constructors.** They can — since C# 10, parameterless constructors with custom logic are allowed.
- **I should use structs for DTOs and entity classes.** No. DTOs and entities are almost always classes. Structs are for small value concepts — coordinates, money, colour,

identifiers.

Practice Problem

Define a readonly record struct called `DateRange` with:

- `Start (DateTime)` and `End (DateTime)`
- A method `DurationInDays()` that returns the number of days between `Start` and `End`
- A method `Contains(DateTime date)` that returns `true` if the date falls within the range (inclusive)
- Validation in the constructor that `End` must not be before `Start`

Then demonstrate creating two `DateRange` values, checking equality, and calling both methods.

Self-check — you should now be able to explain:

- Why copying a struct is independent but copying a class variable is not
- What readonly struct prevents and why it matters
- The five conditions from .NET guidelines for when a struct is appropriate
- Why "structs are always on the stack" is an oversimplification
- When record struct is the right choice over a plain struct

1.17. Comments and XML Documentation Comments

Builds on: Nothing technical — this is about code communication. But you need enough code context (from everything so far) to write meaningful comments.

What are comments and why do they exist?

Code tells you what is happening. Comments tell you why it's happening, or explain things that the code itself can't express clearly.

The most important thing to understand upfront: good code minimises the need for comments. A well-named method, a meaningful variable name, and a clear structure communicate better than a comment explaining bad code. Comments that describe what the code does are often a sign the code should be restructured. Comments that explain why a decision was made are genuinely valuable.

Single-line comments

```
// This is a single-line comment
int retryLimit = 3; // max attempts before giving up

// BAD comment -- restates what the code obviously does:
int x = x + 1; // increment x by 1

// GOOD comment -- explains WHY, not what:
retryCount++; // increment before check so failed attempt is counted
```


Multi-line comments

```
/*
    This block handles the edge case where a user's session
    expires mid-transaction. We intentionally don't roll back here
    because the payment may have already been captured by the gateway.
    See issue #4521 for full context.
*/
public void HandleExpiredSession(Transaction transaction)
{
    // ...
}
```

Multi-line comments are less common in modern C# — consecutive single-line comments are usually preferred. Block comments appear more often for temporarily disabling code during debugging (though that's a code smell if left in).

What makes a good comment vs a bad comment

Bad — describes what the code does (the code already says this):

```
// Loop through orders
foreach (var order in orders)
{
    // Check if order is cancelled
    if (order.Status == OrderStatus.Cancelled)
    {
        // Skip cancelled orders
        continue;
    }
}
```

Good — explains why, or captures non-obvious context:

```
foreach (var order in orders)
{
    if (order.Status == OrderStatus.Cancelled)
        continue; // cancelled orders are billed separately -- see billing spec v2

    // Payment gateway requires Amount in paise (smallest unit), not rupees
    var amountInPaise = (long)(order.Amount * 100);
    await _gateway.Charge(amountInPaise);
}
```

Good uses for comments:

- Explaining why a non-obvious decision was made
- Referencing a ticket, spec, or external document
- Warning about a gotcha or non-obvious constraint
- Explaining a workaround for a bug in a library or framework
- Marking intentional TODO items (with a ticket number, not vague notes)

```
// TODO(#4891): Replace with proper retry policy once Polly is added
await Task.Delay(1000);
await _service.Retry();
```

XML Documentation Comments

XML doc comments are a structured comment format that tools — Visual Studio, Rider, the dotnet doc toolchain, and Swagger — can read and display as documentation.

They use `///` (triple slash) and XML tags.

Basic structure:

```
/// <summary>
/// Calculates the total price including tax for an order.
/// </summary>
/// <param name="amount">The pre-tax order amount in the base currency.</param>
/// <param name="taxRate">The tax rate as a percentage (e.g. 18 for 18%).</param>
/// <returns>The total amount including tax, rounded to 2 decimal places.</returns>
public decimal CalculateTotal(decimal amount, decimal taxRate)
{
    return Math.Round(amount * (1 + taxRate / 100), 2);
}
```

The core XML tags

— what the method/class/property does. Most important tag. Keep it concise — one to three sentences:

```
/// <summary>
/// Retrieves an order by its unique identifier.
/// Returns null if no order with the given ID exists.
/// </summary>
public Order? GetById(int id) { ... }
```

— describes a parameter. One per parameter:

```
/// <param name="id">The unique order identifier. Must be greater than zero.</param>
/// <param name="includeItems">
/// When true, the returned order includes its line items.
/// Set to false for summary views to reduce payload size.
/// </param>
public Order? GetById(int id, bool includeItems = false) { ... }
```

— describes the return value:

```
/// <returns>
/// The matching order, or null if not found.
/// </returns>
```

— documents exceptions the method can throw:

```
/// <exception cref="ArgumentException">
/// Thrown when <paramref name="id"/> is less than or equal to zero.
/// </exception>
/// <exception cref="OrderNotFoundException">
/// Thrown when no order with the given ID exists.
/// </exception>
public Order GetById(int id) { ... }
```

— additional context, usage notes, or caveats beyond the summary:

```
/// <summary>Sends a password reset email to the user.</summary>
/// <remarks>
/// This method is rate-limited to 3 calls per hour per user.
```

```

/// Subsequent calls within the rate limit window return true
/// without sending a duplicate email.
/// </remarks>
public bool SendPasswordReset(string email) { ... }

```

and — cross-references to other types or members:

```

/// <summary>
/// Cancels an order. See <see cref="RefundOrder"/> for refund processing.
/// </summary>
/// <seealso cref="OrderStatus"/>
public void CancelOrder(int id) { ... }

```

Where XML doc comments matter most

Public APIs and libraries — if other developers (or other teams) consume your code, XML doc comments are essential. They appear in IntelliSense without needing to look at source.

Swagger/OpenAPI documentation — ASP.NET Core can extract XML comments and surface them in the Swagger UI. Enable it in Program.cs:

```

builder.Services.AddSwaggerGen(options =>
{
    var xmlFile = $"{Assembly.GetExecutingAssembly().GetName().Name}.xml";
    var xmlPath = Path.Combine(AppContext.BaseDirectory, xmlFile);
    options.IncludeXmlComments(xmlPath);
});

```

And in your .csproj:

```

<PropertyGroup>
  <GenerateDocumentationFile>true</GenerateDocumentationFile>
</PropertyGroup>

```

Now your controller method XML comments appear directly in the Swagger UI as endpoint documentation — your API is self-documenting.

Controller actions specifically:

```

/// <summary>
/// Retrieves a paginated list of orders for the authenticated user.
/// </summary>
/// <param name="page">Page number, starting from 1.</param>
/// <param name="pageSize">Number of orders per page. Maximum 100.</param>
/// <response code="200">Returns the paginated order list.</response>
/// <response code="400">Invalid pagination parameters.</response>
/// <response code="401">User is not authenticated.</response>
[HttpGet]
[ProducesResponseType(typeof(PagedResult<OrderDto>), StatusCodes.Status200OK)]
[ProducesResponseType(StatusCodes.Status400BadRequest)]
[ProducesResponseType(StatusCodes.Status401Unauthorized)]
public IActionResult GetOrders([FromQuery] int page = 1, [FromQuery] int pageSize = 20)
{
    // ...
}

```

Commenting conventions in .NET teams

- Do document all public members of public types — methods, properties, classes

- Do write summaries for internal types used across multiple parts of the codebase
- Don't document private implementation details that aren't consumed externally — it adds noise
- Don't write comments that will go stale
- Do use `// TODO:` with a ticket reference, not vague `// fix this later`
- Do comment non-obvious algorithmic choices or business rules

Common Misconceptions

- **More comments = better code.** Not true. Comments that explain what the code does are noise. The goal is code clear enough that comments are only needed for the why, not the what.
- **XML doc comments are just for library authors.** In any .NET shop with multiple developers or teams, XML doc comments on shared services and interfaces save significant time. IntelliSense displays them during development — it's live documentation.
- **Commented-out code is harmless.** It's a code smell. Commented-out code should be deleted — version control (Git) preserves the history. Leaving dead code in comments confuses future readers.
- **// TODO comments are fine to leave indefinitely.** They should be linked to a real ticket. A `// TODO` with no reference is likely to remain forever.
- **should describe implementation details.** It should describe behaviour from the caller's perspective — what it does and what it returns. Implementation details belong in if anywhere.

Self-check — you should now be able to explain:

- The difference between a comment that describes what code does vs why — and which is valuable
- What the six most important XML doc tags are and when each is used
- How XML doc comments connect to Swagger documentation in ASP.NET Core
- Why commented-out code is a code smell and what to do instead
- What makes a TODO comment acceptable vs one that's a maintenance liability

1.18. Constants — const vs readonly

Builds on: 1.4 (variables and types — constants are a specialised form of variable), 1.11 (methods — readonly fields are set in constructors), 1.16 (structs — readonly applies there too).

What problem do constants solve?

Without named constants, values that appear multiple times in code are repeated as raw literals:

```
if (password.Length < 8) return "Too short";
if (password.Length > 8) SendWarning();
// ... 15 other places using 8
```

If the requirement changes to 12, you hunt through the codebase for every 8 — some of which might be unrelated. Named constants centralise the value:

```
const int MinPasswordLength = 8;

if (password.Length < MinPasswordLength) return "Too short";
```

Change it once, updated everywhere. More readable, less error-prone.

C# has two mechanisms for this: `const` and `readonly`. They look similar but have meaningfully different rules.

const — compile-time constant

`const` values are resolved at compile time and baked directly into the IL. They cannot change — ever.

```
public class Config
{
    public const int MaxRetries = 3;
    public const string DefaultRole = "Viewer";
    public const double Pi = 3.14159265358979;
}
```

Rules for `const`:

- Must be assigned at declaration — no constructor assignment
- Value must be a compile-time constant — a literal, or an expression made entirely of other constants
- Can only be primitive types and string (int, double, bool, string, char, decimal, enums)
- Implicitly static — accessed via the type, not an instance
- Cannot be `readonly` — they are already immutable by definition

```
// Valid const:
public const int MaxItems = 100;
public const string Prefix = "ORD-";
public const int DoubleMax = MaxItems * 2; // expression of constants -- fine

// Invalid const:
public const DateTime Expiry = DateTime.Now; // runtime value -- not allowed
public const List<string> Tags = new List<string>(); // not a primitive -- not allowed
public const int[] Limits = { 1, 2, 3 }; // not a primitive -- not allowed
```

Accessing `const`:

```
// Accessed on the type -- no instance needed:
int limit = Config.MaxRetries;
string role = Config.DefaultRole;
```

readonly — runtime constant

`readonly` fields are assigned once — either at declaration or in a constructor — and cannot be changed afterwards. The value is determined at runtime.

```
public class OrderService
{
    // Assigned at declaration:
```

```

private readonly string _defaultCurrency = "USD";

// Assigned in constructor:
private readonly IOrderRepository _repo;
private readonly DateTime _serviceStartTime;

public OrderService(IOrderRepository repo)
{
    _repo = repo; // set once in constructor
    _serviceStartTime = DateTime.UtcNow; // runtime value -- fine for readonly
}

public void Process()
{
    _repo = null; // compile error -- readonly after constructor
}
}

```

Rules for readonly:

- Can be assigned at declaration OR in a constructor (but nowhere else)
- Can hold any type — objects, collections, arrays, complex types
- Can hold runtime values (DateTime.Now, new List(), etc.)
- Is NOT implicitly static — each instance can have a different value (unless marked static readonly)
- Can be static readonly for a shared, runtime-determined constant

static readonly — the runtime equivalent of const for complex types

When you need a shared constant that's a complex type (not a primitive), static readonly is the pattern:

```

public class AppDefaults
{
    // const -- primitives only:
    public const int MaxPageSize = 100;
    public const string ApiVersion = "v1";

    // static readonly -- any type, set once at class initialisation:
    public static readonly string[] AllowedCurrencies = { "USD", "EUR", "INR", "GBP" };
    public static readonly DateTime AppLaunchDate = new DateTime(2024, 1, 1);
    public static readonly TimeSpan DefaultTimeout = TimeSpan.FromSeconds(30);
}

// Accessed same as const -- on the type:
int max = AppDefaults.MaxPageSize;
string[] currencies = AppDefaults.AllowedCurrencies;
TimeSpan timeout = AppDefaults.DefaultTimeout;

```

The const compile-time baking problem

This is the most important subtlety of const — and the reason you should prefer static readonly for library/SDK code.

When you use a `const` from another assembly, its value is copied into your assembly at compile time:

```
// In LibraryA.dll:
public class Limits
{
    public const int MaxItems = 100;
}

// In YourApp.dll -- compiled against LibraryA:
int limit = Limits.MaxItems;
// Compiler bakes this in as: int limit = 100;
```

If LibraryA later changes `MaxItems` to 200 and you release a new version without recompiling YourApp, YourApp still uses 100. `static readonly` does not have this problem — the value is read from the DLL at runtime.

Practical rule: Use `const` freely within a single project. For values in libraries consumed by other assemblies, prefer `static readonly`.

Where each appears in real .NET code

`const` — small, universal, never-changing values:

```
public class JwtClaims
{
    public const string UserId    = "uid";
    public const string UserRole = "role";
    public const string Email    = "email";
}

// In a controller:
var userId = User.FindFirst(JwtClaims.UserId)?.Value;
```

`readonly` — dependency injection, per-instance configuration:

```
// The single most common pattern in ASP.NET Core:
public class UserService
{
    private readonly IUserRepository _repo;
    private readonly ILogger<UserService> _logger;

    public UserService(IUserRepository repo, ILogger<UserService> logger)
    {
        _repo    = repo;
        _logger = logger;
    }
}
```

Every injected dependency in every service you write follows this pattern. `readonly` enforces that dependencies assigned in the constructor can never be accidentally reassigned elsewhere in the class.

`static readonly` — shared configuration, allowed values, singleton-like data:

```
public class OrderValidator
{
    private static readonly HashSet<string> ValidStatuses = new()
```

```

{
    "Pending", "Processing", "Shipped", "Delivered", "Cancelled"
};

private static readonly TimeSpan MaxOrderAge = TimeSpan.FromDays(365);

public bool IsValid(Order order)
{
    return ValidStatuses.Contains(order.Status)
        && order.CreatedAt > DateTime.UtcNow - MaxOrderAge;
}
}

```

Comparison table

	const	readonly	static readonly
When set	Compile time	Runtime (ctor/decl)	Runtime (static)
Types allowed	Primitives	Any type	Any type
Instance or static	Always static	Per instance	Always static
Can hold new Obj()	No	Yes	Yes
Can hold DateTime	No	Yes	Yes
Baked into caller	Yes (risk)	No	No
Use case	Simple const	DI deps, per-inst	Shared complex

Common Misconceptions

- **const and static readonly are interchangeable.** For primitives within a single project, yes — they behave the same at runtime. For cross-assembly consumption, they differ critically (baking problem). For complex types, only static readonly works.
- **readonly means the object it references can't change.** It means the field can't be reassigned to point to a different object. The object itself can still be mutated. `private readonly List _items` prevents `_items = new List()` but allows `_items.Add("x")`.
- **const can be any type if I just try.** No — the compiler enforces that const values are compile-time computable. Any runtime type (`DateTime`, `List`, custom classes) will be a compile error.
- **I need readonly on all fields to be safe.** You need it on fields that should not change after construction — mainly injected dependencies. Overusing readonly on every field regardless of intent adds noise.
- **static readonly and a static property with only a getter are the same.** A property getter can compute a new value each call. static readonly is assigned once at class initialisation. Use the field for true constants, a property for computed values.

Practice Problem

You're building an `EmailService` class. Define the following:

- A const for the maximum subject line length (250 characters)
- A const for the default sender name ("NoReply")
- A static readonly for the list of blocked domains ("spam.com", "fake.org")

- A static readonly for a default timeout (TimeSpan of 10 seconds)
- A readonly field for an injected ISmtpClient dependency

Then write a method `bool IsValidRecipient(string email)` that:

- Returns false if the email is null or whitespace
- Returns false if the email's domain is in the blocked list
- Returns true otherwise

Self-check — you should now be able to explain:

- Why `const` cannot hold a `DateTime` or `List` but `static readonly` can
- What "baked into the caller" means for `const` and why it matters for library code
- Why `readonly` on a `List` field doesn't prevent calling `.Add()` on it
- The pattern of private `readonly IService _service` in constructors and why `readonly` is appropriate there
- When you'd choose `static readonly` over `const` for a string value

1.19. Scope and Variable Lifetime

Builds on: 1.4 (variables — scope determines where they're accessible), 1.11 (methods — each method has its own scope), 1.16 (structs/classes — fields have class-level scope), 1.18 (`const/readonly` — their scope rules follow from this topic).

What is scope?

Scope is the region of code where a variable is accessible. Outside its scope, the variable simply doesn't exist — trying to use it is a compile error, not a runtime error. The compiler enforces scope, not the runtime.

Lifetime is related but different — it's how long the variable's memory actually exists. Scope determines visibility. Lifetime determines existence.

Block scope — the fundamental rule

In C#, a variable's scope is the block `{ }` it's declared in, from the point of declaration to the closing brace.

```
{
    int x = 10;           // x comes into scope here
    Console.WriteLine(x); // accessible
}
Console.WriteLine(x);    // compile error -- x is out of scope
```

Every pair of `{ }` creates a new scope. This applies to methods, if blocks, loops, try/catch blocks — any braces.

Variable scope in practice

Method scope:

```

public void ProcessOrder(int orderId)
{
    // orderId is in scope for the entire method
    var order = _repo.GetById(orderId); // order declared here

    if (order.Status == OrderStatus.Pending)
    {
        var message = "Processing..."; // message only exists in this if block
        Console.WriteLine(message);
    }

    // message is NOT accessible here -- out of scope
    Console.WriteLine(order.Status); // order is still in scope
}

```

Loop scope:

```

for (int i = 0; i < 5; i++)
{
    int doubled = i * 2; // doubled is scoped to each loop iteration
    Console.WriteLine(doubled);
}

// i and doubled are NOT accessible here
Console.WriteLine(i); // compile error
Console.WriteLine(doubled); // compile error

```

Nested scope — inner can see outer, not vice versa

An inner scope can access variables from the outer scope. The outer scope cannot access variables from the inner scope.

```

int outerValue = 10; // outer scope

if (true)
{
    int innerValue = 20; // inner scope
    Console.WriteLine(outerValue); // inner can see outer
    Console.WriteLine(innerValue); // in scope
}

Console.WriteLine(outerValue); // still in scope
Console.WriteLine(innerValue); // compile error -- outer cannot see inner

```

Variable shadowing — what C# prevents

Some languages allow declaring a variable in an inner scope with the same name as an outer scope variable. C# prevents this for local variables to protect against bugs:

```

int x = 10;

if (true)
{
    int x = 20; // compile error -- x already declared in enclosing scope
}

```

Exception — class fields can be shadowed by local variables:

```

public class OrderService
{
    private int _count = 0;    // field

    public void Process()
    {
        int _count = 5; // local variable shadows the field -- compiles but is confusing
        Console.WriteLine(_count);    // prints 5 -- local variable wins

        // Access the field explicitly with this:
        Console.WriteLine(this._count);    // prints 0 -- the field
    }
}

```

This compiles but is considered bad practice. The convention of prefixing fields with `_` (as in `_count`) is partly to make this shadowing visually obvious and avoidable.

Class-level scope — fields and properties

```

public class OrderService
{
    private readonly IOrderRepository _repo;    // class scope
    private int _processedCount = 0;            // class scope
    public string ServiceName = "OrderService"; // class scope (public)

    public void Process(Order order)
    {
        // All class-level members accessible here:
        var result = _repo.Save(order);
        _processedCount++;
        Console.WriteLine($"{ServiceName}: processed {_processedCount}");
    }

    public void Reset()
    {
        // Also accessible here -- class scope spans all methods:
        _processedCount = 0;
    }
}

```

Static scope — shared across all instances

```

public class RequestTracker
{
    private static int _totalRequests = 0;    // shared across all instances

    public void TrackRequest()
    {
        _totalRequests++;    // increments the shared counter
    }

    public static int GetTotal() => _totalRequests;
}

var t1 = new RequestTracker();

```

```
var t2 = new RequestTracker();

t1.TrackRequest();
t2.TrackRequest();

Console.WriteLine(RequestTracker.GetTotal()); // 2 -- both tracked in same counter
```

Variable lifetime vs scope

Scope is about visibility (compile time). Lifetime is about memory existence (runtime).

Local variables:

- Scope: the block they're declared in
- Lifetime: allocated when the block is entered, eligible for garbage collection when the block exits

```
public void ProcessOrder()
{
    var helper = new ExpensiveHelper(); // allocated on heap, reference on stack
    helper.DoWork();
}
// helper goes out of scope here
// the reference is gone -- GC can now free the ExpensiveHelper object
```

Field lifetime:

- Scope: accessible within the class
- Lifetime: as long as the object containing it is alive (referenced somewhere)

Static field lifetime:

- Scope: accessible via the type
- Lifetime: for the entire duration of the application — static fields never get garbage collected while the app is running

This is why static fields that accumulate data (like caches or lists) can cause memory leaks if not managed — they're never collected.

Scope in closures — variables captured by lambdas

Lambda expressions (used heavily in LINQ and async code) can capture variables from their enclosing scope — this is called a closure:

```
int multiplier = 3;

Func<int, int> multiply = x => x * multiplier; // multiplier is captured

Console.WriteLine(multiply(5)); // 15

multiplier = 10; // change the captured variable
Console.WriteLine(multiply(5)); // 50 -- uses current value, not value at capture time
```

The lambda doesn't capture the value of multiplier at the time it's created — it captures a reference to the variable. If the variable changes, the lambda sees the new value.

Classic loop capture bug:

```
var actions = new List<Action>();
```

```

for (int i = 0; i < 3; i++)
{
    actions.Add(() => Console.WriteLine(i)); // captures i -- not its current value
}

actions[0](); // prints 3 -- not 0!
actions[1](); // prints 3 -- not 1!
actions[2](); // prints 3 -- not 2!

```

By the time the lambdas run, the loop has finished and *i* is 3. All three lambdas captured the same *i* variable, not a snapshot of its value.

Fix — capture a local copy inside the loop:

```

for (int i = 0; i < 3; i++)
{
    int captured = i; // new variable per iteration -- each lambda gets its own
    actions.Add(() => Console.WriteLine(captured));
}

actions[0](); // 0
actions[1](); // 1
actions[2](); // 2

```

In foreach — C# 5+ fixed this:

```

foreach (var item in items)
{
    actions.Add(() => Console.WriteLine(item)); // safe in foreach -- each iteration has its own
}

```

foreach iteration variables are scoped per iteration in C# 5 and later. The for loop bug still applies because *i* is a single variable shared across all iterations.

Realistic scenario — scope in a controller action

```

[HttpPost("checkout")]
public async Task<IActionResult> Checkout([FromBody] CheckoutDto dto)
{
    var result = await _validator.ValidateAsync(dto);

    if (!result.IsValid)
    {
        // errors scoped to this if block:
        var errors = result.Errors.Select(e => e.ErrorMessage);
        return BadRequest(errors);
    }

    // errors not accessible here -- out of scope

    var order = await _orderService.CreateAsync(dto);

    if (order.RequiresApproval)
    {
        // approvalNote scoped to this if block:
        var approvalNote = $"Order {order.Id} pending approval";
        await _notificationService.Notify(approvalNote);
        return Accepted(new { order.Id, message = approvalNote });
    }
}

```

```

    }

    return Ok(order);
}

```

Common Misconceptions

- **Variables declared at the top of a method are best practice.** This is a habit from C and older C++. In C#, declare variables at the point of first use — the narrowest scope possible.
- **Going out of scope immediately frees memory.** For value types on the stack, yes. For reference types, going out of scope makes the object eligible for garbage collection — but the GC decides exactly when memory is freed.
- **Lambdas capture the value of a variable, not the variable itself.** They capture the variable — a reference to it. If the variable changes after the lambda is created, the lambda sees the new value. This is the source of the classic loop capture bug.
- **Static fields are garbage collected when no instance references them.** Static fields are never garbage collected while the application runs. They live for the entire process lifetime.
- **Two variables in different scopes can never have the same name in C#.** Local variables in enclosing scopes can't share names with inner scope locals. But two variables in completely separate sibling scopes (two different if blocks at the same level) can share names without issue.

```

if (condition1)
{
    string message = "First";    // fine
}

if (condition2)
{
    string message = "Second";  // also fine -- completely separate scope
}

```

Self-check — you should now be able to explain:

- What "scope" means and at what point the compiler enforces it
- Why for loop variables aren't accessible after the loop
- What a closure captures — the variable or its value — and why the loop capture bug happens
- Why static fields can cause memory leaks but instance fields don't have the same risk
- Why declaring variables at the narrowest possible scope is preferred in C#

1.20. var and Implicit Typing

Builds on: 1.4 (types — var is still statically typed), 1.5 (type conversion — var doesn't change types), 1.19 (scope — var follows the same scope rules).

What is var and why does it exist?

`var` tells the compiler to infer the type from the right-hand side of the assignment. The type is still fixed at compile time — `var` is not dynamic typing. It's just the compiler figuring out the type so you don't have to write it twice.

```
// These are identical at compile time:
string name = "Sathish";
var name = "Sathish";    // compiler infers: this is a string

List<Order> orders = new List<Order>();
var orders = new List<Order>();    // compiler infers: List<Order>
```

The compiled IL is identical in both cases. `var` is purely a compile-time convenience — there is zero runtime difference.

How the compiler infers the type

The type is inferred from whatever is on the right side of `=`. It's always a specific, concrete type — never ambiguous:

```
var i      = 42;           // int
var d      = 3.14;         // double
var m      = 3.14m;        // decimal
var s      = "hello";      // string
var b      = true;         // bool
var order  = new Order();  // Order
var orders = new List<Order>(); // List<Order>
var now    = DateTime.UtcNow; // DateTime
```

Hover over any `var` in Visual Studio or Rider — the IDE shows you the inferred type. You're never in the dark about what type a variable is.

Where `var` is unambiguously the right choice

LINQ queries — the return type is complex and redundant to write:

```
// Without var -- verbose and redundant:
IEnumerable<IGrouping<string, Order>> grouped =
    orders.GroupBy(o => o.Status);

// With var -- clean, type is obvious from context:
var grouped = orders.GroupBy(o => o.Status);

// Another LINQ example:
var activeUserNames = users
    .Where(u => u.IsActive)
    .Select(u => u.Name)
    .OrderBy(n => n)
    .ToList();
// Type is List<string> -- inferred cleanly
```

Anonymous types — `var` is the only option:

```
// Anonymous types have no writable type name -- var is required:
var summary = new { Name = "Sathish", OrderCount = 5, Total = 999.99m };
Console.WriteLine(summary.Name);           // "Sathish"
Console.WriteLine(summary.OrderCount);     // 5
```

`foreach` with complex element types:

```
// Without var:
foreach (KeyValuePair<string, List<Order>> entry in ordersByStatus)

// With var:
foreach (var entry in ordersByStatus)    // cleaner, type is clear from the collection name
```

Where var reduces clarity — avoid it

When the right side doesn't reveal the type:

```
// What type is result? You have to look at GetResult's signature:
var result = GetResult();

// Better -- explicit type makes it immediately clear:
OrderResult result = GetResult();

// Is this int, long, double, decimal?
var timeout = GetTimeout();

// Explicit -- the type is part of the information:
TimeSpan timeout = GetTimeout();
```

The practical guideline used by most .NET teams

Most codebases and style guides settle on something close to:

Use var when the type is apparent from the right-hand side or when the exact type is unimportant (LINQ, anonymous types, foreach). Use explicit types when the type is important for understanding the code or not obvious from context.

The Roslyn team's rule (the C# compiler team themselves): use var when the right-hand side is a new expression, a cast, or a literal — because the type is stated right there. Use explicit types for method return values where the type isn't visible without looking elsewhere.

var is NOT dynamic

```
var x = 10;        // x is int -- permanently
x = "hello";       // compile error -- can't assign string to int
x = 3.14;          // compile error -- can't assign double to int
```

var fixes the type at compile time. It is not JavaScript's var, Python's dynamic typing, or C#'s dynamic. Once inferred, the type cannot change.

Compare with dynamic — which IS dynamic:

```
dynamic d = 10;
d = "hello";    // allowed -- dynamic bypasses compile-time type checking
d = 3.14;       // allowed
```

dynamic defers type checking to runtime. var is just compile-time type inference. They are fundamentally different. In everyday .NET application code, dynamic is rare — var is common.

var with target-typed new — C# 9+

C# 9 introduced target-typed new — the mirror image of var. Instead of inferring the type from the right side, the compiler infers it from the left side:

```
// var -- type inferred from right side:
```



```
var orders = new List<Order>();
```

```
// Target-typed new -- type inferred from left side:  
List<Order> orders = new(); // new() instead of new List<Order>()
```

Both approaches reduce redundancy — you write the type once instead of twice. Modern .NET codebases use both:

```
// Common in modern C#:  
List<string> names = new();  
Dictionary<int, Order> orderMap = new();  
OrderService service = new(repository, logger); // constructor args still work
```

For local variables, `var` is often cleaner. For fields and properties (where the type is declared separately), target-typed `new` is natural:

```
public class OrderService  
{  
    private readonly List<Order> _cache = new(); // field -- target-typed new  
    private readonly Dictionary<int, string> _map = new(); // field  
  
    public void Process()  
    {  
        var orders = new List<Order>(); // local -- var  
        var result = GetResult(); // local -- var (if type is clear)  
    }  
}
```

Realistic scenario — `var` in a service method

```
public async Task<OrderSummaryDto> GetSummaryAsync(int userId)  
{  
    // var -- type obvious from method name and new expression:  
    var user = await _userRepo.GetByIdAsync(userId);  
  
    if (user == null)  
        throw new UserNotFoundException(userId);  
  
    // var -- complex LINQ type, not worth spelling out:  
    var recentOrders = await _orderRepo  
        .GetByUserAsync(userId)  
        .Where(o => o.CreatedAt > DateTime.UtcNow.AddDays(-30))  
        .OrderByDescending(o => o.CreatedAt)  
        .ToListAsync();  
  
    // var -- type obvious from new expression:  
    var summary = new OrderSummaryDto  
    {  
        UserId      = userId,  
        UserName    = user.Name,  
        OrderCount  = recentOrders.Count,  
        TotalSpent  = recentOrders.Sum(o => o.Amount)  
    };  
  
    return summary;  
}
```

Common Misconceptions

- **var is dynamic typing.** It is not. var is compile-time type inference. The type is fixed at the point of declaration and enforced by the compiler just as strictly as an explicit type declaration.
- **var makes code less readable.** When the type is obvious from context, var has zero effect on readability. When the type is not obvious, explicit types are better.
- **Using var everywhere is best practice.** No style guide recommends var everywhere without consideration. Use it where it adds clarity or reduces redundancy; use explicit types where the type is part of the information.
- **var is slower at runtime.** There is zero runtime difference. The IL generated from var int and int is identical. var only affects compile time.
- **var x = null infers the type as object.** It doesn't compile — the compiler cannot infer a type from null alone. var x = null is a compile error.

Practice Problem

Look at this code and identify where var is appropriate, where explicit types are better, and why:

```
var i = 0;
var name = "Sathish";
var result = Process();
var orders = new List<Order>();
var query = orders.Where(o => o.Amount > 100).GroupBy(o => o.Status);
var flag = true;
var timeout = GetTimeout();
var dto = new CreateOrderDto { Amount = 500, Currency = "USD" };
```

For each line, state: keep var, change to explicit, or depends — and explain your reasoning. There is no single correct answer for every line — this is about articulating the reasoning.

Self-check — you should now be able to explain:

- Why var is not dynamic typing and what happens if you try to reassign a different type
- In which two scenarios var is always the right choice (LINQ and anonymous types)
- When explicit types communicate more than var would
- What target-typed new() is and how it differs from var
- Why var x = null is a compile error

1.21. Pattern Matching — is, switch expressions, property patterns

Builds on: 1.7 (switch expressions — you've already seen basic switch patterns), 1.4 (types — pattern matching heavily involves type checking), 1.14 (nullable types — null patterns are part of this).

What is pattern matching and why does it exist?

Before pattern matching, testing what something is and then using it required repetitive, verbose code:

```
// Old approach -- check type, cast, then use:
object shape = GetShape();

if (shape is Circle)
{
    Circle c = (Circle)shape;    // cast again after checking
    Console.WriteLine(c.Radius);
}
else if (shape is Rectangle)
{
    Rectangle r = (Rectangle)shape;
    Console.WriteLine(r.Width * r.Height);
}
```

Pattern matching collapses the check-and-cast into a single operation, and extends that idea to checking properties, ranges, combinations, and more — all with compile-time safety.

The **is** pattern — type check and cast in one

```
object shape = new Circle { Radius = 5 };

if (shape is Circle c)
{
    // c is Circle -- already cast, immediately usable:
    Console.WriteLine(c.Radius);    // 5
}
```

Compare with the old two-step approach — **is** with a pattern variable eliminates the redundant cast entirely.

Works with null safety:

```
object obj = null;

if (obj is Circle c)
{
    // Never enters here -- null does not match any type pattern
}
```

is never matches a type pattern — the check is null-safe by design.

Negation — **is not**:

```
if (shape is not Circle)
{
    Console.WriteLine("Not a circle");
}

// Also works with type and variable:
if (shape is not null)
{
    Console.WriteLine("Not null");
}
```

is not reads naturally and is preferred over **!(shape is Circle)** in modern C#.

Patterns in switch expressions — the full picture

You've seen basic switch expressions in 1.7. Pattern matching makes them dramatically more powerful.

Type patterns in switch:

```
public double GetArea(object shape) => shape switch
{
    Circle c      => Math.PI * c.Radius * c.Radius,
    Rectangle r   => r.Width * r.Height,
    Triangle t    => 0.5 * t.Base * t.Height,
    null          => throw new ArgumentNullException(nameof(shape)),
    _             => throw new ArgumentException($"Unknown shape: {shape.GetType().Name}")
};
```

Property patterns — match on property values

```
public string ClassifyOrder(Order order) => order switch
{
    { Status: "Cancelled" }                => "Cancelled order",
    { Status: "Pending", Amount: > 10000 } => "High-value pending",
    { Status: "Pending" }                  => "Standard pending",
    { Status: "Shipped", IsExpress: true } => "Express shipment",
    { Status: "Shipped" }                  => "Standard shipment",
    _                                       => "Other"
};
```

The `{ PropertyName: pattern }` syntax matches if the property satisfies the given pattern. Multiple properties in one arm work as AND — all must match.

Nested property patterns:

```
public string GetDeliveryMessage(Order order) => order switch
{
    { Customer: { IsPremium: true }, Status: "Shipped" }
      => "Priority delivery en route",
    { Customer: { Country: "India" }, Status: "Shipped" }
      => "Domestic delivery in progress",
    { Status: "Shipped" }
      => "International delivery in progress",
    _      => "Not yet shipped"
};
```

Relational patterns — comparisons in patterns

```
public string GetRiskLevel(decimal amount) => amount switch
{
    <= 0      => throw new ArgumentException("Amount must be positive"),
    < 1000    => "Low",
    < 10000   => "Medium",
    < 100000  => "High",
    _        => "Critical"
};

public string ClassifyTemperature(double celsius) => celsius switch
{
    ...
};
```

```

    < 0    => "Freezing",
    < 10   => "Cold",
    < 20   => "Cool",
    < 30   => "Warm",
    _     => "Hot"
};

```

Logical patterns — and, or, not in patterns

```

// and -- both conditions must match:
public bool IsValidScore(int score) => score is >= 0 and <= 100;

// or -- either condition matches:
public bool IsWeekend(DayOfWeek day) =>
    day is DayOfWeek.Saturday or DayOfWeek.Sunday;

// not -- inverts the pattern:
public bool IsWorkday(DayOfWeek day) =>
    day is not (DayOfWeek.Saturday or DayOfWeek.Sunday);

// Combined in a switch:
public string GetScoreCategory(int score) => score switch
{
    < 0 or > 100    => "Invalid",
    >= 90           => "Excellent",
    >= 70 and < 90   => "Good",
    >= 50 and < 70   => "Pass",
    _              => "Fail"
};

```

Note the keywords: and, or, not (lowercase) — these are pattern keywords, not the &&, ||, ! operators. They only work inside pattern contexts.

List patterns — C# 11

Match on the structure of a list or array:

```

int[] numbers = { 1, 2, 3 };

bool result = numbers switch
{
    []           => true,    // empty array
    [1, 2, 3]    => true,    // exactly these three values
    [1, ..]      => true,    // starts with 1 (.. matches any remaining elements)
    [.., 3]      => true,    // ends with 3
    [_ , _ , _] => true,    // exactly three elements of any value
    _           => false
};

```

Practical example — parsing CLI args:

```

string[] args = { "run", "--port", "5000" };

string message = args switch
{
    ["run"]           => "Running with defaults",
    ["run", "--port", var port] => $"Running on port {port}",
};

```

```

        ["stop"]                => "Stopping",
        _                       => "Unknown command"
    };
    // "Running on port 5000"

```

when guards — conditional patterns

Add a when condition to a pattern arm — the arm only matches if both the pattern AND the condition are true:

```

public string ProcessOrder(Order order) => order switch
{
    { Status: "Pending" } when order.Amount > 50000
        => "Requires manual approval",

    { Status: "Pending" } when _fraudService.IsSuspicious(order)
        => "Flagged for review",

    { Status: "Pending" }
        => "Auto-approved",

    _ => "Not pending"
};

```

when is evaluated after the pattern matches — it's a filter on top of the structural match. Use it for conditions that can't be expressed as pure patterns (method calls, complex calculations).

Deconstruction patterns — with tuples and custom types

```

// Tuple deconstruction in a pattern:
(int x, int y) point = (3, 4);

string quadrant = point switch
{
    (> 0, > 0) => "First quadrant",
    (< 0, > 0) => "Second quadrant",
    (< 0, < 0) => "Third quadrant",
    (> 0, < 0) => "Fourth quadrant",
    _         => "On an axis"
};

// HTTP status classification:
(int statusCode, string method) request = (404, "GET");

string response = request switch
{
    (200, _)      => "OK",
    (404, "GET")  => "Resource not found",
    (404, _)      => "Endpoint not found",
    (>= 500, _)   => "Server error",
    _             => "Other"
};

```

Realistic scenario — domain rule engine in a .NET API

```

public class PricingService
{
    public decimal CalculateDiscount(Order order, Customer customer) =>
        (order, customer) switch
        {
            // Premium customers get extra discount on large orders:
            ({ Amount: > 10000 }, { IsPremium: true })
                => order.Amount * 0.15m,

            // All customers get discount on large orders:
            ({ Amount: > 10000 }, _)
                => order.Amount * 0.10m,

            // Premium customers always get some discount:
            (_, { IsPremium: true })
                => order.Amount * 0.05m,

            // New customers get a welcome discount:
            (_, { OrderCount: 0 })
                => order.Amount * 0.03m,

            // No discount:
            _ => 0m
        };
}

```

Common Misconceptions

- **Pattern matching is just syntactic sugar for if/else.** At a basic level yes, but it adds genuine capabilities: exhaustiveness checking, simultaneous type-check-and-cast, and structural matching that can't be cleanly expressed with if/else chains.
- **and, or, not in patterns work like &&, ||, !.** They have the same logical meaning but only work inside pattern contexts. You can't use && in a switch expression arm, and you can't use and in a regular boolean expression.
- **Property patterns check for exact equality only.** Property patterns support any nested pattern — { Amount: > 1000 } uses a relational pattern inside a property pattern.
- **null matches a type pattern.** It never does. obj is Order o is false when obj is null. If you need to handle null explicitly, add a null => arm to your switch.
- **Pattern matching is only for switch expressions.** is patterns work in if conditions, while conditions, and anywhere a boolean expression is valid.

Practice Problem

Write a method DescribeValue(object? value) using a switch expression with patterns that returns:

- "null" if value is null
- "negative number: X" if value is a negative int or double
- "positive number: X" if value is a positive int or double
- "zero" if value is int 0 or double 0.0

- "short text" if value is a string with 10 or fewer characters
- "long text" if value is a string with more than 10 characters
- "boolean: true/false" if value is a bool
- "collection of N items" if value is a List
- "unknown: TypeName" for everything else

Self-check — you should now be able to explain:

- What `Circle c` does in a single expression and why it's better than the old two-step approach
- How property patterns work and how to match on multiple properties in one arm
- The difference between `and/or/not` keywords in patterns vs `&&/||/!` operators
- Why `null` never matches a type pattern and how to handle it explicitly
- What a `when` guard adds to a pattern arm and when to use it

1.22. Tuples and Deconstruction

Builds on: 1.4 (value types — tuples are value types), 1.11 (methods — tuples solve the "return multiple values" problem), 1.21 (pattern matching — tuples are deconstructed in patterns).

What problem do tuples solve?

Methods can only return one value. When you need to return two or three related values, the options before tuples were:

```
// Option 1 -- out parameters (verbose, awkward):
public bool TryGetUser(int id, out string name, out string email) { ... }

// Option 2 -- create a class just to carry data (over-engineered for simple cases):
public class UserLookupResult { public string Name; public string Email; }
public UserLookupResult GetUser(int id) { ... }

// Option 3 -- tuples (clean, no ceremony for simple cases):
public (string name, string email) GetUser(int id) { ... }
```

Tuples give you a lightweight way to group a small number of related values without defining a class.

Tuple syntax — creating tuples

```
// Value tuple -- the modern C# tuple (C# 7+):
var point = (10, 20); // unnamed elements
var point = (X: 10, Y: 20); // named elements

// Explicit type declaration:
(int X, int Y) point = (10, 20);
(string name, int age) person = ("Sathish", 30);
```

Named vs unnamed elements:

```
var unnamed = (10, 20);
```



```

Console.WriteLine(unnamed.Item1);    // 10 -- generic name
Console.WriteLine(unnamed.Item2);    // 20

var named = (X: 10, Y: 20);
Console.WriteLine(named.X);           // 10 -- meaningful name
Console.WriteLine(named.Y);           // 20

```

Always use named elements in practice. Item1, Item2 are unreadable — they communicate nothing about what the values mean.

Tuples as return types

```

public (bool success, string error) ValidateEmail(string email)
{
    if (string.IsNullOrEmpty(email))
        return (false, "Email is required");

    if (!email.Contains('@'))
        return (false, "Email must contain @");

    return (true, string.Empty);
}

// Calling it:
var result = ValidateEmail("sathish@zoho.com");

if (!result.success)
    return BadRequest(result.error);

```

Deconstruction — unpacking tuples into individual variables

```

var (success, error) = ValidateEmail("s@zoho.com");

if (!success)
    Console.WriteLine(error);

```

The `var (a, b) = tuple` syntax assigns each element to a new variable. The names on the left don't have to match the tuple element names — they're just new variable names you're choosing.

Explicit types in deconstruction:

```

(bool success, string error) = ValidateEmail("s@zoho.com");

```

Discarding elements you don't need:

```

var (success, _) = ValidateEmail("s@zoho.com");
// _ discards the error -- you only care about success

```

Deconstruction in foreach

```

var coordinates = new List<(int X, int Y)>
{
    (1, 2), (3, 4), (5, 6)
};

foreach (var (x, y) in coordinates)
{
    Console.WriteLine($"X={x}, Y={y}");
}

```

```
}
```

Deconstruction with existing variables

```
int x, y;  
(x, y) = (10, 20);    // assign to existing variables -- no var
```

Custom deconstruction — Deconstruct method

Your own classes can support deconstruction by defining a Deconstruct method:

```
public class Order  
{  
    public int Id { get; set; }  
    public string Status { get; set; }  
    public decimal Amount { get; set; }  
  
    public void Deconstruct(out int id, out string status, out decimal amount)  
    {  
        id = Id;  
        status = Status;  
        amount = Amount;  
    }  
}
```

```
// Now Order can be deconstructed:  
var order = new Order { Id = 42, Status = "Shipped", Amount = 999.99m };  
  
var (id, status, amount) = order;  
Console.WriteLine($"Order {id}: {status} -- {amount:C}");  
// "Order 42: Shipped -- $999.99"
```

Records and record structs generate Deconstruct automatically:

```
public record Order(int Id, string Status, decimal Amount);  
  
var order = new Order(42, "Shipped", 999.99m);  
var (id, status, amount) = order;    // works automatically
```

Tuple equality

```
var a = (1, "hello");  
var b = (1, "hello");  
var c = (2, "hello");  
  
Console.WriteLine(a == b);    // true  
Console.WriteLine(a == c);    // false  
  
// Element names don't affect equality:  
var p1 = (X: 1, Y: 2);  
var p2 = (Row: 1, Col: 2);  
  
Console.WriteLine(p1 == p2);    // true -- names ignored for equality
```

When to use tuples vs classes/records

Use tuples when:

- Returning 2-3 values from a private or internal method
- The grouping is temporary — used immediately and not stored long-term
- The values are obvious from context — no need for a named type
- The consumer is close to the producer (same class or file)

Use a class or record when:

- The data has a meaningful name as a concept (OrderSummary, ValidationResult)
- It's returned from a public API surface
- It's stored in a collection or passed across multiple methods
- It has behaviour (methods) or needs XML doc comments

```
// Tuple -- private helper, used immediately:
private (decimal subtotal, decimal tax) CalculateAmounts(Order order)
{
    var subtotal = order.Items.Sum(i => i.Price);
    var tax = subtotal * 0.18m;
    return (subtotal, tax);
}

// Record -- public API return type, meaningful concept:
public record OrderTotals(decimal Subtotal, decimal Tax, decimal GrandTotal);

public OrderTotals GetOrderTotals(Order order)
{
    var (subtotal, tax) = CalculateAmounts(order);
    return new OrderTotals(subtotal, tax, subtotal + tax);
}
```

Tuples in LINQ

```
var orderSummaries = orders
    .Where(o => o.IsActive)
    .Select(o => (o.Id, o.CustomerName, Total: o.Items.Sum(i => i.Price)))
    .OrderByDescending(o => o.Total)
    .ToList();

foreach (var (id, customer, total) in orderSummaries)
{
    Console.WriteLine($"Order {id} | {customer} | {total:C}");
}
```

Realistic scenario — service method returning multiple outcomes

```
public class PaymentService
{
    public async Task<(bool success, string transactionId, string error)>
        ProcessPaymentAsync(decimal amount, string cardToken)
    {
        if (amount <= 0)
            return (false, string.Empty, "Amount must be positive");
        try
```

```

    {
        var txId = await _gateway.ChargeAsync(amount, cardToken);
        return (true, txId, string.Empty);
    }
    catch (PaymentDeclinedException ex)
    {
        return (false, string.Empty, ex.Message);
    }
    catch (GatewayTimeoutException)
    {
        return (false, string.Empty, "Payment gateway timeout -- please retry");
    }
}
}

// In the controller:
var (success, transactionId, error) = await _paymentService.ProcessPaymentAsync(amount, token);

if (!success)
    return BadRequest(new { error });

return Ok(new { transactionId });

```

Common Misconceptions

- **Tuples are the same as anonymous types.** They're different. Anonymous types are reference types, cannot be returned from methods as a named type, and are primarily for LINQ projections within a method. Tuples are value types, can be return types, and are usable across method boundaries.
- **Named tuple elements are enforced at runtime.** Element names are compile-time only — at runtime, tuples only have Item1, Item2 etc. Names are interchangeable because the runtime structure is identical.
- **Tuples should replace classes for all data grouping.** No. Public API return types, stored data, and meaningful domain concepts deserve named types. Tuples are for lightweight, local, short-lived grouping.
- **Deconstruction only works on tuples.** Any type with a Deconstruct method can be deconstructed — including your own classes, records, and KeyValuePair. Records get Deconstruct automatically.
- **(int, string) and ValueTuple are different.** They're the same type — (int, string) is shorthand syntax for ValueTuple. The C# compiler translates the tuple syntax to ValueTuple under the hood.

Practice Problem

Write a method ParseCsvLine(string line) that:

- Takes a CSV line like "42,Sathish Kumar,999.99,true"
- Returns a named tuple (int id, string name, decimal amount, bool isActive)
- If parsing fails for any field, returns (0, string.Empty, 0m, false) as a sentinel

Then write a loop that processes these lines:

```
var lines = new[]
{
    "1,Sathish,1500.00,true",
    "2,Priya,750.50,false",
    "bad,Karthik,200.00,true",
    "4,Ravi,abc,true"
};
```

Deconstruct each result in the foreach and print valid records (where id > 0). Try before moving on.

Self-check — you should now be able to explain:

- Why named tuple elements should always be used over Item1, Item2
- What deconstruction does and how the `_` discard works in it
- When to use a tuple vs a class or record — give a concrete example of each
- How to add deconstruction support to your own classes
- Why `(X: 1, Y: 2) == (Row: 1, Col: 2)` is true

1.23. Namespaces and using Directives

Builds on: 1.2 (project structure — namespaces map to folder structure), 1.1 (BCL — all BCL types live in namespaces like `System`, `System.Collections.Generic`).

What are namespaces and why do they exist?

As a codebase grows, name collisions become inevitable. Two different libraries might both define a class called `Order`, `Logger`, or `Result`. Without namespaces, using both in the same project would be impossible.

Namespaces are containers that scope type names. `MyApp.Domain.Order` and `ThirdParty.Models.Order` are completely distinct types — the namespace is part of the full name.

Two things namespaces solve:

- Name collision avoidance — types in different namespaces can share names
- Organisation — namespaces communicate where a type lives and what it's for

Defining a namespace

```
namespace MyApp.Domain.Entities
{
    public class Order
    {
        public int Id { get; set; }
        public string Status { get; set; }
    }
}
```

The fully qualified name of this class is `MyApp.Domain.Entities.Order`. Without a `using` directive, you'd have to write that full name every time you use it.

File-scoped namespaces — C# 10+ (the modern default):

```
namespace MyApp.Domain.Entities;    // semicolon, no braces

public class Order
{
    public int Id { get; set; }
    public string Status { get; set; }
}
```

File-scoped namespaces apply to the entire file — no braces, one level less of indentation. All dotnet new templates in .NET 6+ use this style. Prefer it for all new code.

Namespace conventions in .NET projects

Namespaces almost always follow the folder structure:

```
MyApp/
+-- MyApp.API/
|   +-- Controllers/
|   |   +-- OrdersController.cs    -> namespace MyApp.API.Controllers
|   +-- Program.cs                 -> namespace MyApp.API
+-- MyApp.Application/
|   +-- Services/
|   |   +-- OrderService.cs        -> namespace MyApp.Application.Services
|   +-- DTOs/
|   |   +-- CreateOrderDto.cs      -> namespace MyApp.Application.DTOs
+-- MyApp.Domain/
|   +-- Entities/
|   |   +-- Order.cs               -> namespace MyApp.Domain.Entities
```

using directives — importing namespaces

Without using, every reference to a BCL type requires the full name:

```
System.Collections.Generic.List<System.String> names =
    new System.Collections.Generic.List<System.String>();
```

With using:

```
using System.Collections.Generic;

List<string> names = new List<string>();
```

using imports all public types from the specified namespace into the current file — you can then use their short names directly.

Common namespaces you'll use daily

```
using System;                // DateTime, Console, Math, Exception
using System.Collections.Generic; // List<T>, Dictionary<TKey,TValue>, HashSet<T>
using System.Linq;           // LINQ extension methods
using System.Threading.Tasks; // Task, Task<T>
using System.Text;           // StringBuilder
using System.Text.Json;      // JsonSerializer
using Microsoft.AspNetCore.Mvc; // ControllerBase, IActionResult, [HttpGet]
using Microsoft.EntityFrameworkCore; // DbContext, DbSet<T>
using Microsoft.Extensions.Logging; // ILogger<T>
```

Implicit usings — .NET 6+

Modern .NET projects with enable in the .csproj automatically import the most commonly needed namespaces. You don't write these yourself — they're injected by the SDK:

For a Web API project, these are implicitly included:

```
// Automatically available -- no using directive needed:
using System;
using System.Collections.Generic;
using System.IO;
using System.Linq;
using System.Net.Http;
using System.Threading;
using System.Threading.Tasks;
using Microsoft.AspNetCore.Builder;
using Microsoft.AspNetCore.Hosting;
using Microsoft.AspNetCore.Http;
using Microsoft.AspNetCore.Routing;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Hosting;
using Microsoft.Extensions.Logging;
```

This is why modern dotnet new webapi projects have almost no using statements at the top of files — they're handled by the SDK. You add using only for namespaces not in the implicit set.

using aliases — rename a type for the file

When two namespaces both contain a type with the same name, use an alias to disambiguate:

```
using MyOrder = MyApp.Domain.Entities.Order;
using ApiOrder = ThirdPartyLibrary.Models.Order;

MyOrder domainOrder = new MyOrder { Id = 1 };
ApiOrder externalOrder = new ApiOrder { OrderId = "EXT-001" };
```

Aliases are file-scoped — they only apply in the file where they're declared.

Global using directives — C# 10+

global using applies to the entire project — not just one file. Put them in a dedicated file (conventionally GlobalUsings.cs):

```
// GlobalUsings.cs
global using System;
global using System.Collections.Generic;
global using System.Linq;
global using Microsoft.AspNetCore.Mvc;
global using MyApp.Domain.Entities;
global using MyApp.Application.Services;
```

Every file in the project can now use these types without any using directive. This reduces boilerplate across the codebase.

When to use global usings:

- Namespaces used in almost every file in the project

- Your own domain entities and core types

When not to:

- Namespaces that are only relevant in a few files — global usings can confuse readers who see a type used with no visible import

Static using — import static members

Import static members of a class so you can call them without the class name:

```
using static System.Math;
using static System.Console;

double result = Sqrt(144);    // Math.Sqrt without "Math."
WriteLine(result);           // Console.WriteLine without "Console."
```

Common legitimate use — in test files:

```
using static Xunit.Assert;

Equal(expected, actual);      // instead of Assert.Equal(...)
True(condition);              // instead of Assert.True(...)
```

Namespace structure in a real ASP.NET Core solution

```
// MyApp.Domain/Entities/Order.cs
namespace MyApp.Domain.Entities;

public class Order
{
    public int Id { get; set; }
    public decimal Amount { get; set; }
    public OrderStatus Status { get; set; }
}

// MyApp.Application/Services/OrderService.cs
namespace MyApp.Application.Services;

using MyApp.Domain.Entities;
using MyApp.Application.Interfaces;

public class OrderService : IOrderService
{
    public Order GetById(int id) { ... }
}

// MyApp.API/Controllers/OrdersController.cs
namespace MyApp.API.Controllers;

using Microsoft.AspNetCore.Mvc;
using MyApp.Application.Services;
using MyApp.Domain.Entities;

[ApiController]
[Route("api/orders")]
public class OrdersController : ControllerBase
{

```



```

private readonly IOrderService _orderService;

public OrdersController(IOrderService orderService)
{
    _orderService = orderService;
}

[HttpGet("{id}")]
public IActionResult GetById(int id)
{
    var order = _orderService.GetById(id);
    return order is null ? NotFound() : Ok(order);
}
}

```

The namespace of each file reflects its layer and folder. using directives bring in types from other layers. The dependency direction flows: API -> Application -> Domain (never backwards).

Common Misconceptions

- **Namespaces and folders are the same thing.** They're related by convention but are independent concepts. A file in Controllers/ folder doesn't automatically get a .Controllers namespace — the namespace is just a string in the file. The convention of matching them exists to make navigation predictable.
- **using imports the code — like copying it in.** using only makes short names available. The actual types are in their assemblies — using just tells the compiler where to look when it sees an unqualified name. No code is copied.
- **Implicit usings mean I never need to write using directives.** Implicit usings cover the most common BCL namespaces. You still need explicit using for third-party libraries (EF Core, Serilog, AutoMapper) and your own project namespaces.
- **global using should be used for everything.** Only for namespaces that are genuinely used throughout the project. A global using for a namespace used in two files adds noise everywhere.
- **File-scoped namespaces are a limitation.** A file-scoped namespace applies to the entire file and supports multiple class definitions in the same file. The only thing you can't do is have two different namespaces in the same file.

Self-check — you should now be able to explain:

- What a fully qualified type name is and why namespaces make it necessary
- The difference between using, global using, and using static
- What implicit usings are and why modern projects have almost no using directives in individual files
- How namespaces map to folder structure by convention and why that convention exists
- When using aliases are useful and how to write one

Section 1 complete. All 23 topics from 1.1 through 1.23 are covered. This PDF covers topics 1.11 through 1.23. The previous PDF covers 1.1 through 1.10. Say next to begin Section 2:

Object-Oriented Programming in C#.